

Kubernetes K8s Notes

Notes of K8s Learning

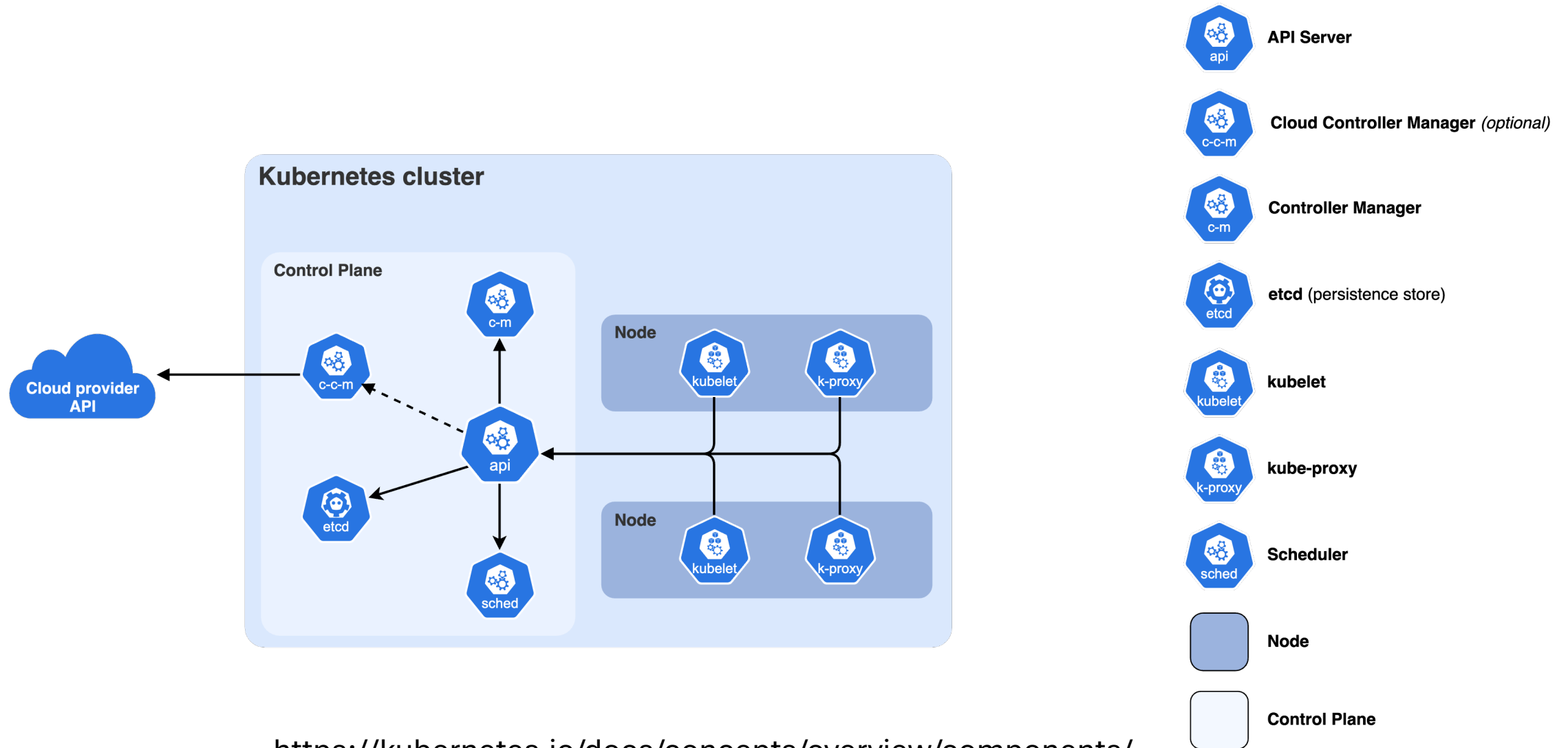
What K8s can do

- Service discovery and load balancing
- Storage orchestration
 - Local or Cloud based
- Automated rollouts and rollbacks
- Automatic bin packing
- Horizontal scaling
- Batch Execution
- Self-healing
- Secrets and configuration management
- Use the same API across on-premise and every cloud providers

What K8s can't do

- Does not deploy source code
- Does not build your application
- Does not provide application-level services
 - Message buses, databases, caches, etc

K8s Architecture



<https://kubernetes.io/docs/concepts/overview/components/>

Local K8s

- Requires virtualization
 - Docker Desktop
 - MicroK8s
 - Minikube
- Runs Over Docker Destop
 - Kind
- Limited to 1 Node
 - Docker Desktop
- Multiple Nodes
 - MicroK8s
 - Kind
 - Minikube

Local K8s – Windows

- Docker Desktop is currently the only way to run both the Linux and Windows containers
- Docker Desktop can run on Hyper-V or WSL 2 (Win 10 2004 or later)
- If Hyper-V is enabled, you can't run another hypervisor at the same time
- You can install Minikube on Hyper-V or VirtualBox

Running k8s using docker desktop

- Tick enable in the kubernetes from docker desktop
- Use command `kubectl cluster-info` in terminal to check status

```
% kubectl cluster-info
```

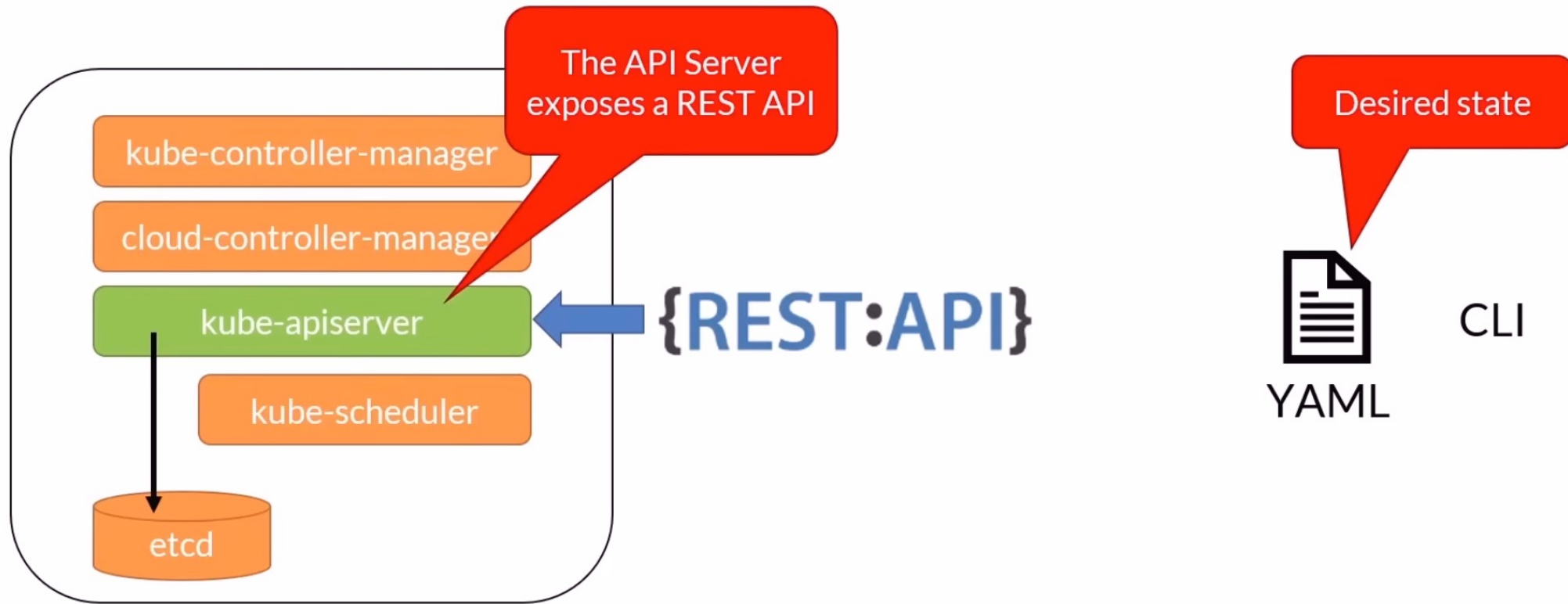
```
Kubernetes control plane is running at https://127.0.0.1:6443
```

```
CoreDNS is running at https://127.0.0.1:6443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy
```

```
To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
```

```
eric@Mac-Air: ~ %
```

K8s API



CLI

- kubectl
- Pronounced (pick your choice)
 - Kube control
 - Kube kuttle
 - Kube ctl
- Communicates with the apiserver
- Configuration stored locally
 - `${HOME}/.kube/config`
 - `C:\Users\{USER}\.kube\config`

K8s Context

- A context is a group of access parameters to a K8s cluster
- Contains a Kubernetes cluster, a user, and a namespace
- The current context is the cluster that is currently the default for kubectl
 - All kubectl commands run against that cluster

kubectl – Context Cheat Sheet

Command	Brief Description
<code>kubectl config current-context</code>	Get the current context
<code>kubectl config get-contexts</code>	List all context
<code>kubectl config use-context [contextName]</code>	Set the current context
<code>kubectl config delete-context [contextName]</code>	Delete a context from the config file

Kubectx – Quickly switch context

- Instead of typing
 - `kubectl config use-context minikube`
- Simply type
 - `kubectx [contextName]`
- Windows
 - `choco install kubectx-ps`
- macOS
 - `brew install kubectx`
- Ubuntu
 - `sudo apt install kubectx`
- <https://github.com/ahmedtb/kubectx>

The declarative way vs the imperative way

- Imperative
 - Using kubectl commands, issue a series of commands to create resources
 - Great for learning, testing and troubleshooting
 - It's like code
- Declarative
 - Using kubectl and YAML manifests defining the resources that you need
 - Reproducible, repeatable
 - Can be saved in source control
 - It's like data that can be parsed and modified

Example

Series of commands

```
kubectl run mynginx --image=nginx --port=80

kubectl create deploy mynginx --image=nginx --
port=80 --replicas=3

kubectl create service nodeport myservice --
targetPort=8080

kubectl delete pod nginx
```

YAML File

```
apiVersion: v1
kind: pod
metadata:
  name: myapp-pod
  labels: myapp
  type: front-end
spec:
  containers:
    - name: nginx-container
      image:nginx
```

YAML

- Root level required properties
 - apiVersion
 - Api version of the object
 - kind
 - type of object
 - metadata.name
 - unique name for the object
 - metadata.namespace
 - scoped environment name (will default to current)
 - spec
 - object specifications or desired state
 - Create an object using YAML
 - `kubectl create -f [YAML file]`

Pod definition

```
apiVersion: v1
kind: pod
metadata:
  name: myapp-pod
  labels: myapp
  type: front-end
spec:
  containers:
    - name: nginx-container
      image:nginx
```

Creating new manifests

- Copy from kubernetes.io
- Use template from VS code
- Use kubectl using the following command

```
kubectl create deploy mynginx --image=nginx --port=80 --replicas=3 --dry-run=client -o yaml
```

Namespaces

- Allow to group resources
 - Ex: Dev, Test, Prod
- K8s creates a default workspace
- Objects in one namespace can access objects in a different one.
 - Ex: `objectname.prod.svc.cluster.local`
- Deleting a namespace will delete all its child objects

Namespaces

- You define a namespace
- You specify the namespace when defining objects

Namespace Definition

```
apiVersion: v1
kind: Namespace
metadata:
  name: prod
```

Pod Definition

```
apiVersion: v1
kind: Namespace
metadata:
  name: myapp-prod
  namespace: prod
spec:
  containers:
    - name: nginx-
      container
        image: nginx
```

Namespaces

NetworkPolicy Definition

```
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata:
  namespace: clientb
  name: deny-from-other-namespaces
spec:
  podselector:
    matchLabels:
  ingress:
    - from:
      - podSelector: {}
```

ResourceQuota Definition

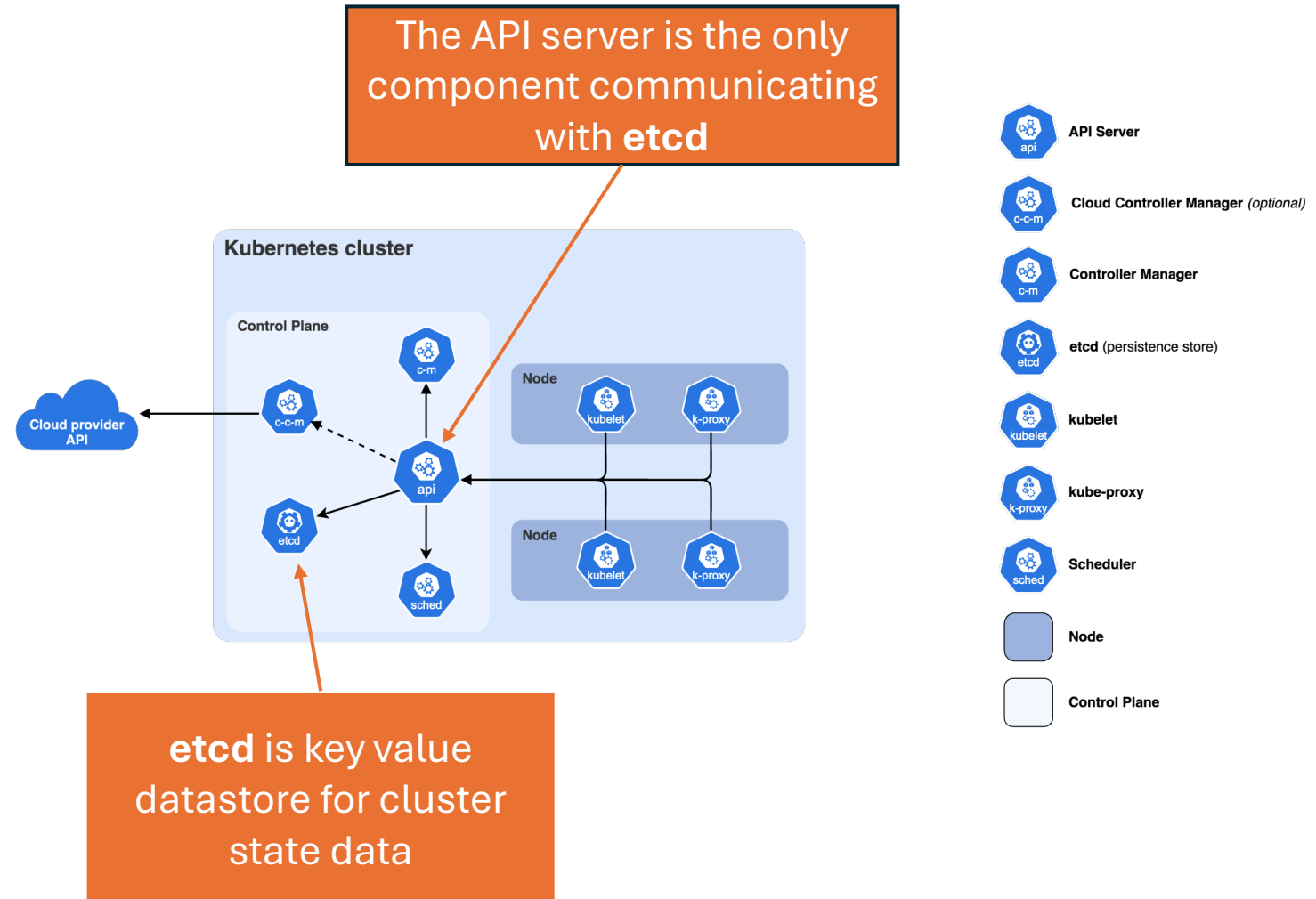
```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-quota
  namespace: prod
spec:
  hard:
    pods: "10"
    limits.cpu:"5"
    limits.memory: 10Gi
```

kubectl – Namespace Cheat Sheet

Command	Brief Description
<code>kubectl get namespace</code>	List all namespaces
<code>kubectl get ns</code>	Shortcut for List all namespaces
<code>kubectl config set-context --current --namespace=[namespaceName]</code>	Set the current context to use a namespace
<code>kubectl create ns [namespaceName]</code>	Create a namespace
<code>kubectl delete ns [namespaceName]</code>	Delete a namespace
<code>kubectl get pods --all-namespaces</code>	List all pods in all namespaces

K8s Architecture – Master Node/Control Plane

- Nodes are physical or virtual machines
- Together they form cluster
- Master Node is the one which controls the other nodes. It is also known as control plane
- K8s services and controllers are in master node. Also they are known as master components/control plane components
- You don't run containers in master node.



kube-apiserver (api)

- Rest interface
- Save state to the datastore(etcd)
- All clients interact with it, never directly to the datastore
- The core component server that exposes the Kubernetes HTTP API.

etcd

- Acts as the cluster datastore for storing state
- Consistent and highly-available key value store for all API server data.
- Not a database or a datastore for applications to use
- The single source of truth

kube-control-manager (c-m)

- Runs controllers to implement Kubernetes API behavior. The controller of the controllers
- It runs controllers
 - Node controller
 - Replication controller
 - Endpoints controller
 - Service account & Token controllers

cloud-control-manager (c-c-m) -optional

- Integrates with underlying cloud provider(s).
- Interact with the cloud providers controllers
 - Node
 - For checking the cloud provider to determine if a node has been deleted in the cloud after it stops responding
 - Route
 - For setting up routes in the underlying cloud infrastructure
 - Service
 - For creating, updating and deleting cloud provider load balancers
 - Volume
 - For creating, attaching and mounting volumes, and interacting with the cloud provider to orchestrate volumes

kube-scheduler (sched)

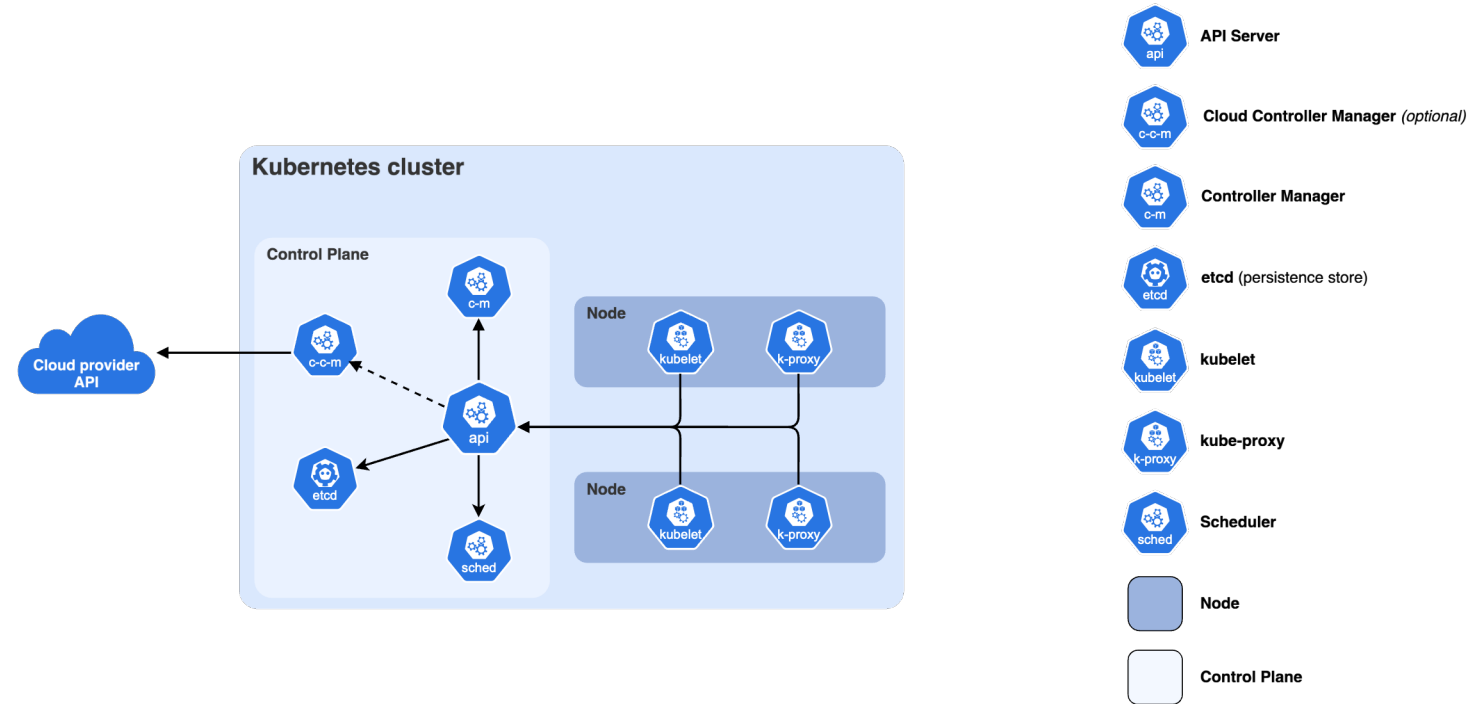
- Looks for Pods not yet bound to a node, and assigns each Pod to a suitable node.
- Watches newly created pods that have no node assigned, and selects a node for them to run on
- Factors taken into account for scheduling decisions include
 - Individual and collective resource requirements
 - Hardware/software/policy constraints
 - Affinity and anti-affinity specifications
 - Data locality

Addons

- Addons extend the functionality of Kubernetes. A few important examples include:
- DNS
 - For cluster-wide DNS resolution.
- Web UI (Dashboard)
 - For cluster management via a web interface.
- Container Resource monitoring
 - For collecting and storing container metrics.
- Cluster-level Logging
 - For saving container logs to a central log store.

K8s Architecture - Nodes

- Nodes are physical or virtual machines
- Together they form cluster
- Master Node is the one which controls the other nodes. It is also known as control plane
- The nodes running the containers are called the worker nodes
- When a worker node is added to a cluster node components are added automatically to the node. These components are necessary to run the pods.
- Node Components: **kubelet**, **kube-proxy**(optional), **container runtime**



kubelet

- Manage the pods lifecycle
- Ensures that the containers described in the Pod specs are running and healthy
- Ensures that Pods are running, including their containers.

kube-proxy (optional)

- A network proxy
- Manages network rules on nodes
- Maintains network rules on nodes to implement services

Container runtime

- Software responsible for running containers
- K8s supports several container runtimes
- Must implement the Kubernetes Container Runtime Interface
 - Moby
 - Containerd
 - Cri-O
 - Rkt
 - Kata
 - Virtlet

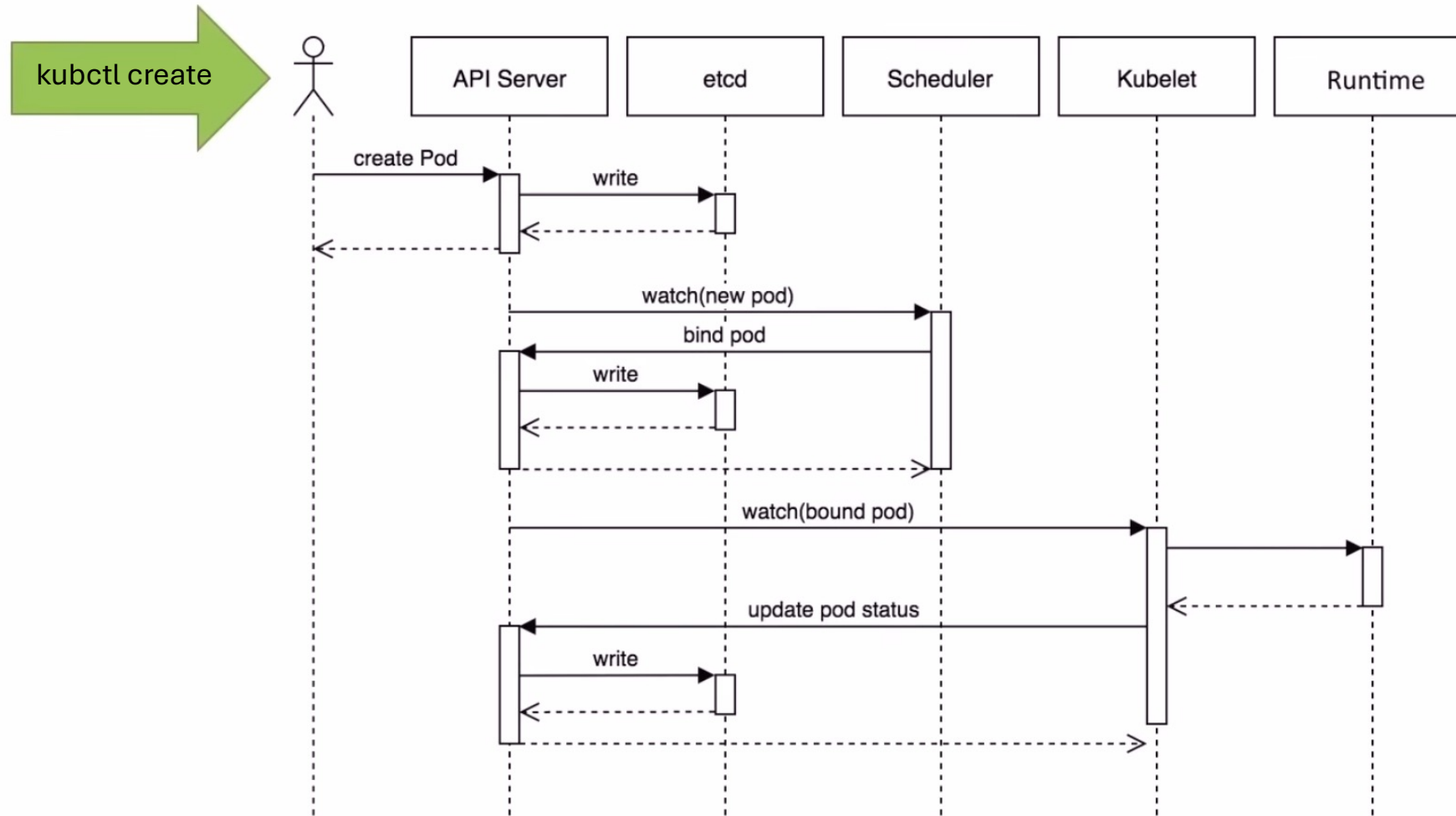
Nodes Pool

- A node pool is a group of virtual machines, all with the same size
- A cluster can have multiple node pools
 - These pools can host different sizes of VMs
 - Each pool can be autoscaled independently from the other pools
- Docker desktop is limited to 1 node

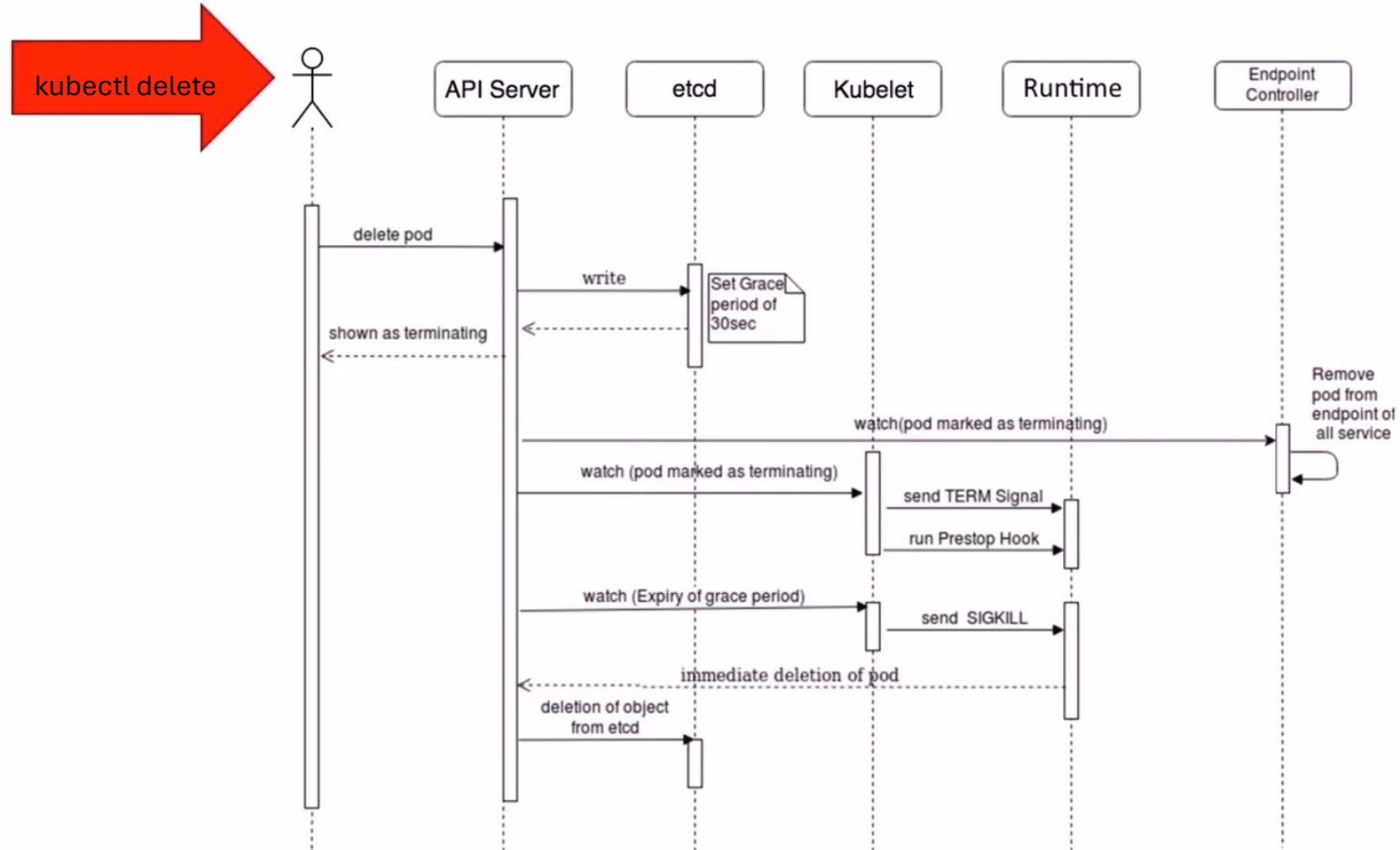
Pods

- Atomic unit of the smallest unit of work of K8s
- Encapsulates an application's container
- Represents a unit of deployment
- Pods can run one or multiple containers
- Containers within a pod share
 - IP address space, mounted volumes
- Containers within a pod can communicate via
 - Localhost, IPC
- Pods are ephemeral
- Deploying a pod is an atomic operation, it succeed or not
- If a pod fails it is replaced with a new one with a shiny new IP address
- You don't update a pod, you replace it with an updated version
- You scale by adding more pods, not more containers in a pod

Pod Creation



Pod deletion



Pod state

- Pending
 - Accepted but not yet created
- Running
 - Bound to a node
- Succeeded
 - Exited with status 0
- Failed
 - All containers exit and at least one exited with non-zero status
- Unknown
 - Communication issues with the pod
- CrashLoopBackOff
 - Started, crashed, started again, and then crashed again

kubectl – Pod Cheat Sheet

Command	Brief Description
<code>kubectl create -f [pod-definition.yaml]</code>	Create a pod
<code>kubectl run [podname] --image=busybox -- /bin/sh -c "sleep 3600"</code>	Run a pod
<code>kubectl get pods</code>	List the running pods
<code>kubectl get pods -o wide</code>	List the running pods with more info
<code>kubectl describe pod [podName]</code>	Show pod info
<code>kubectl get pod [podName] -o yaml > file.yaml</code>	Extract the pod definition in YAML and save it to a file
<code>kubectl exec -it [podName] -sh</code>	Interactive Mode
<code>kubectl delete -f [pod-definition.yaml]</code>	Delete a pod
<code>kubectl delete pod [podName]</code>	Deleted a pod using its name

Init Containers

- Initialize a Pod before an application container runs
- Always run to completion
- Each init container must complete successfully before the next one starts
- If it fails, the kubelet repeatedly restarts it until it succeeds
 - Unless it's restart policy is set to Never
- Probes are not supported
 - livenessProbe, readinessProbe, or startupProbe

Init Containers

Finally the main container will run

App container

1st will run this

init containers

After succesful running of init-myservice, this will run

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
spec:
  containers:
  - name: myapp-container
    image: busybox
  initContainers:
  - name: init-myservice
    image: busybox:1.28
    command: ['sh', '-c', "until nslookup mysvc.namespace.svc.cluster.local; do echo waiting for myservice; sleep 2; done"]
  - name: init-mydb
    image: busybox:1.28
    command: ['sh', '-c', "until nslookup mydb.namespace.svc.cluster.local; do echo waiting for mydb; sleep 2; done"]
```

Labels

- Key-value pairs used to identify, describe and group related sets of objects or resources

Pod Definition

```
apiVersion: v1
kind: pod
metadata:
  name: myapp-prod
  namespace: prod
  labels:
    app: myapp
    type: front-end
spec:
  containers:
  - name: nginx-container
    image: nginx
```

Selectors

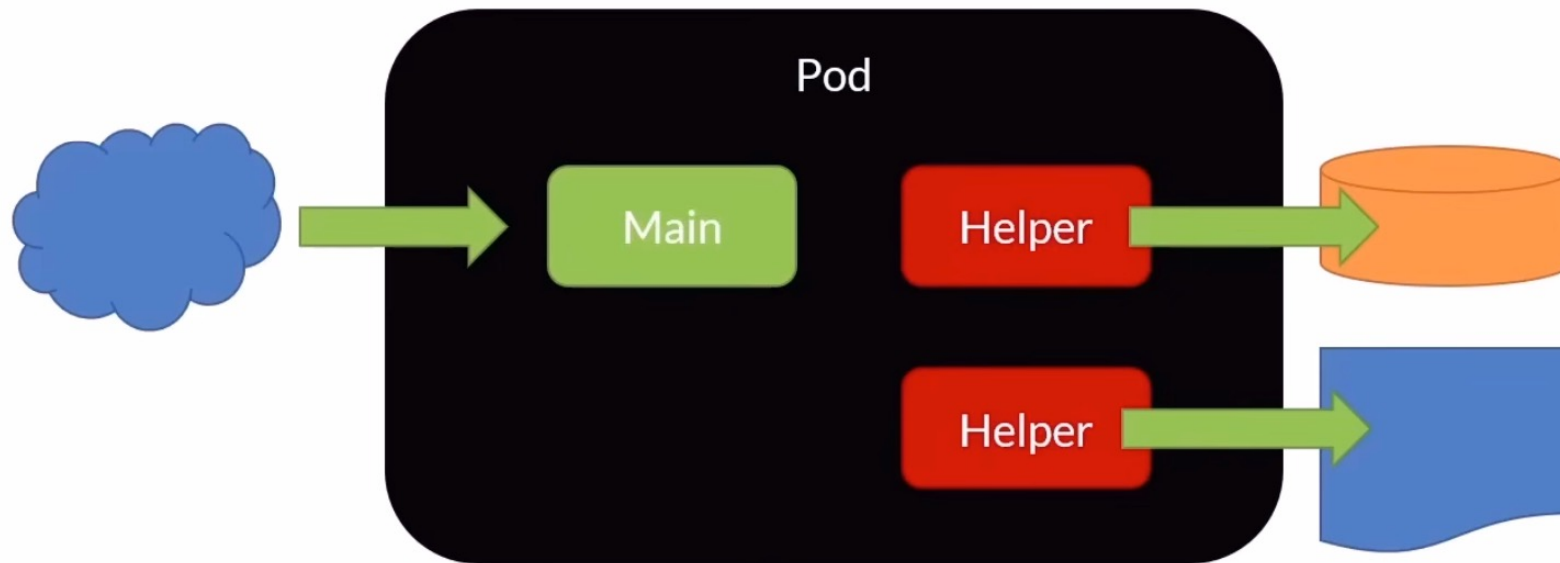
- Selectors use labels to filter or select objects

Pod Definition

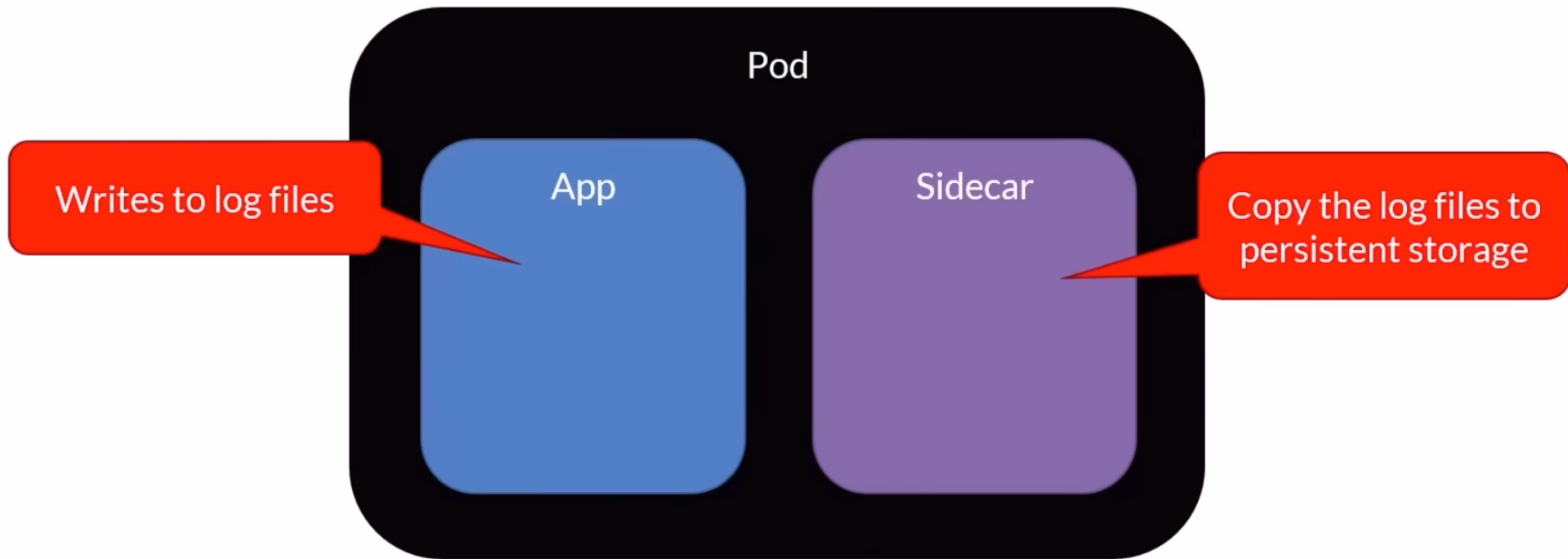
```
apiVersion: v1
kind: pod
metadata:
  name: myapp-prod
  labels:
    app: myapp
    type: front-end
spec:
  containers:
  - name: nginx-container
    image: nginx
  nodeSelector:
    disktype: superfast
```

Multi-container Pods

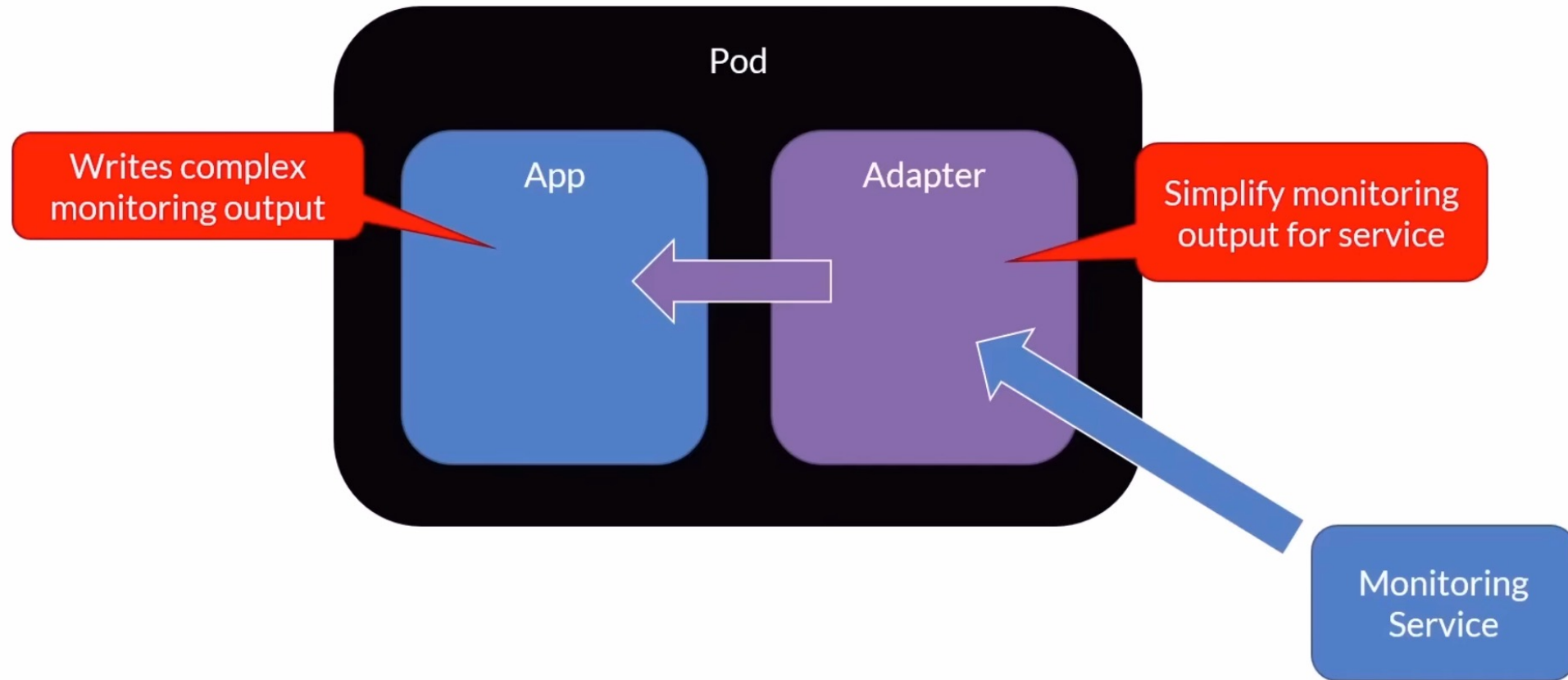
- Most common scenario: Helper process



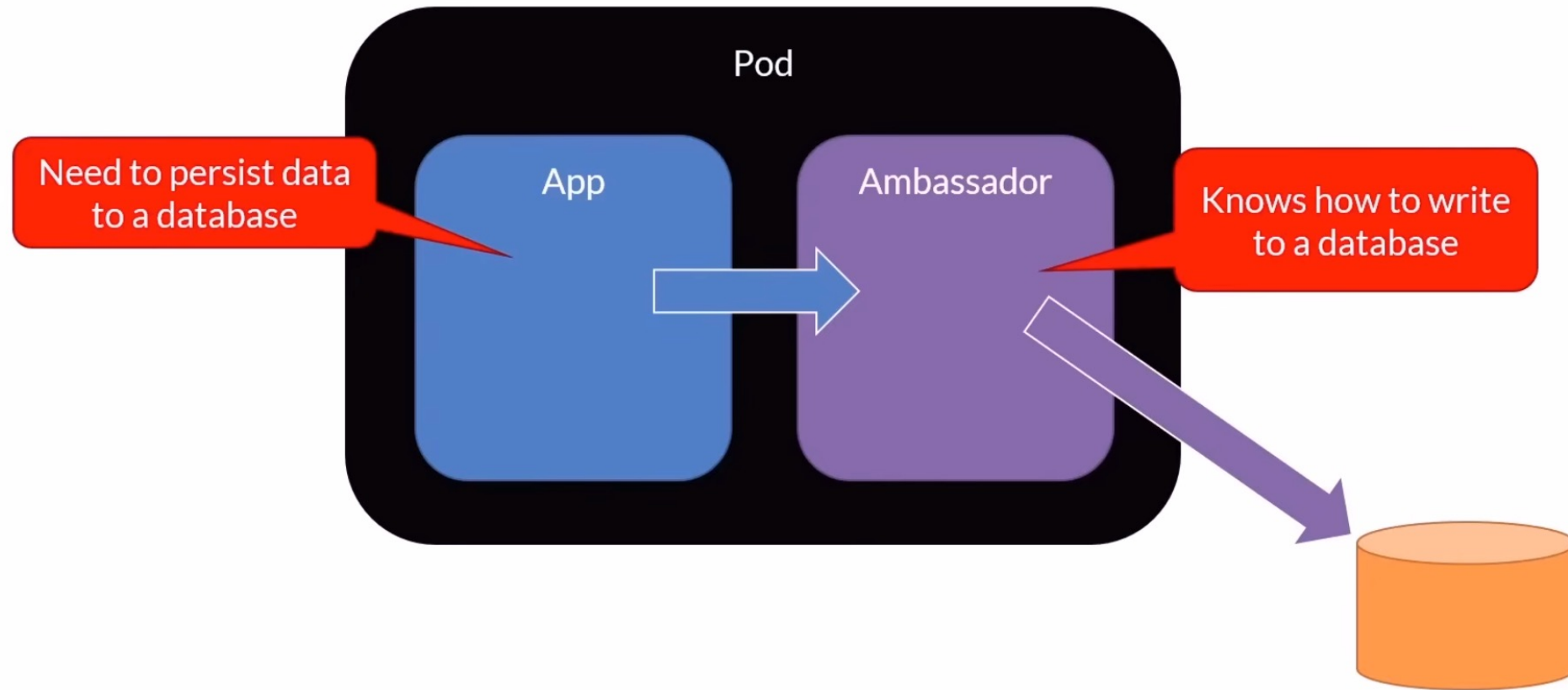
Typical patterns - Sidecar



Typical patterns - Adapter



Typical patterns – Ambassador



Multi-Container Pods

Pod Definition

```
apiVersion: v1
kind: Pod
metadata:
  name: two-containers
spec:
  restartPolicy: Always
  containers:
    - name: mynginx
      image: nginx
      ports:
        - containerPort: 80
    - name: mybox
      image: busybox
      ports:
        - containerPort: 81
      command:
        - sleep
        - "3600"
```

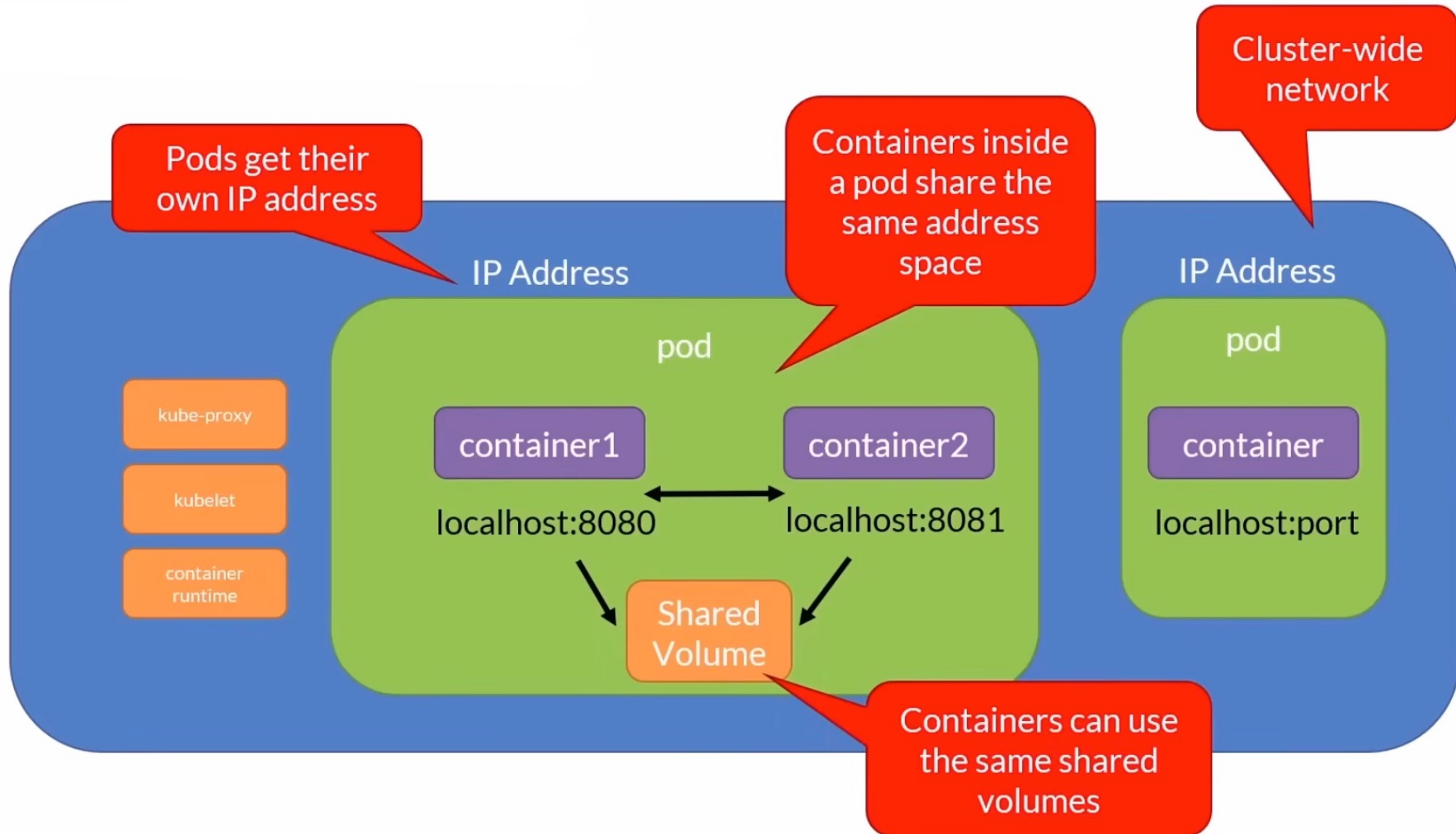
kubectl – Pod Cheat Sheet

Command	Brief Description
<code>kubectl create -f [pod-definition.yml]</code>	Create a pod
<code>kubectl exec -it [podName] -c [containerName] -- sh</code>	Exec into a pod
<code>kubectl logs [podName] -c [containerName]</code>	Get the logs for a container

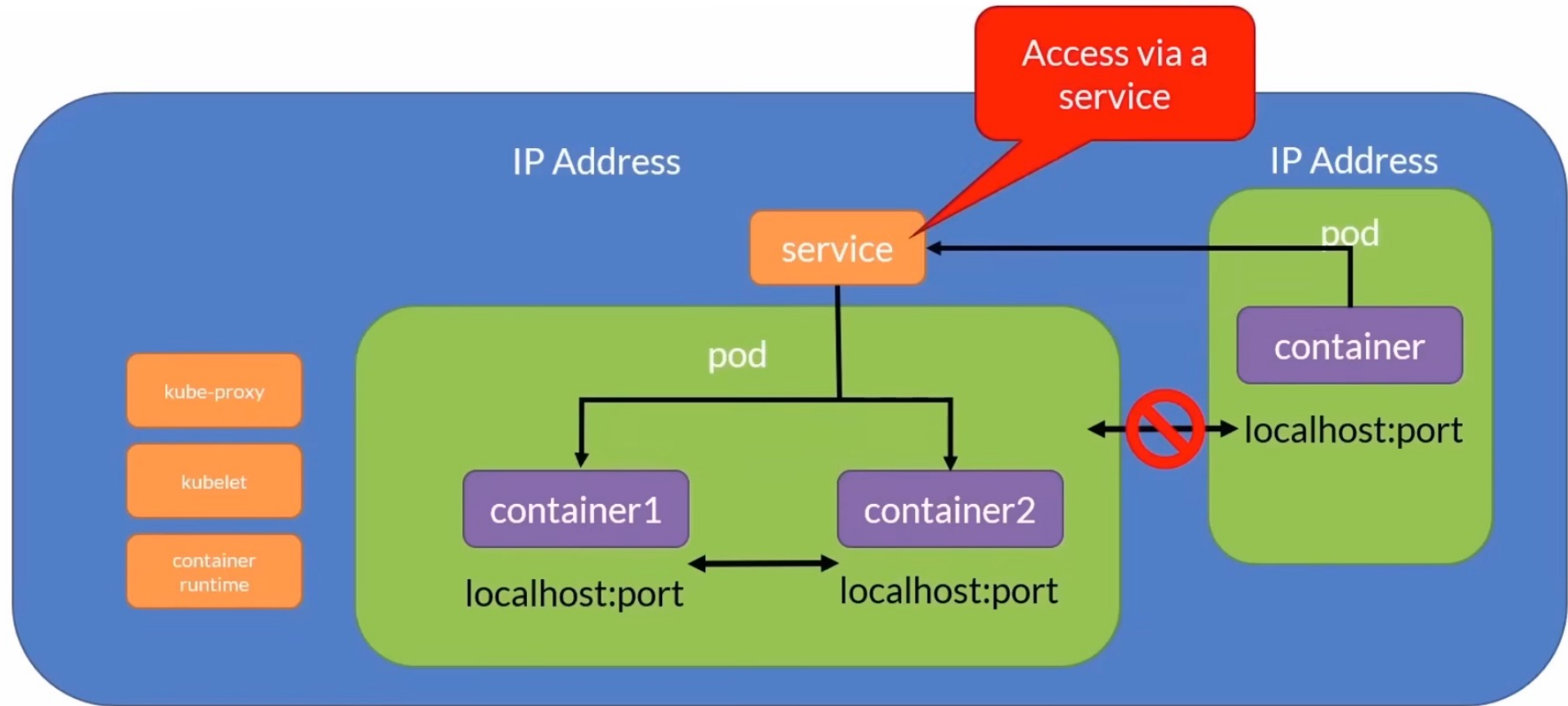
Networking Concepts

- All containers within a pod can communicate with each other
- All pods can communicate with each other
- All nodes can communicate with all pods
- Pods are given an IP address (ephemeral)
- Services are given a persistent IP

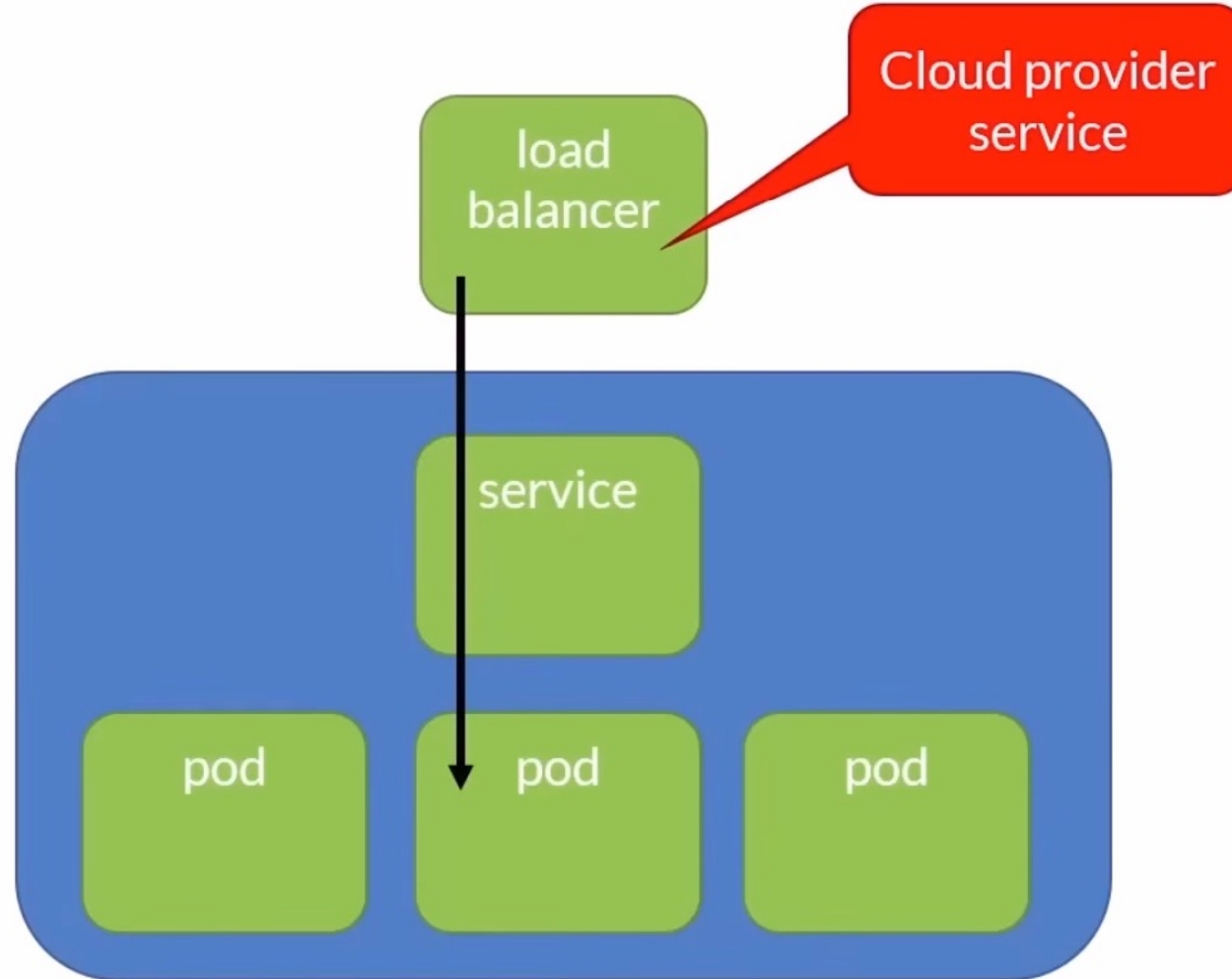
Pod Networking



Inter Pod Communication



External access



Workloads

- A workload is an application running on kubernetes
 - Pod
 - Represents a set of running containers
 - ReplicaSet
 - Deployment
 - Statefulset
 - DeamonSet
 - Provide node-local facilities, such as a storage driver or network plugin
 - Tasks that run to completion
 - Job
 - CronJob

Workloads -ReplicaSets

- Primary method of managing pod replicas and their lifecycle to provide self-healing capabilities
- Their job is to always ensure the desired number of pods are running
- While you can create ReplicaSets, the recommended way is to create Deployments
- Self-healing: If a pod crashes, the pod is replaced automatically.

kubectl – ReplicaSets Cheat Sheet

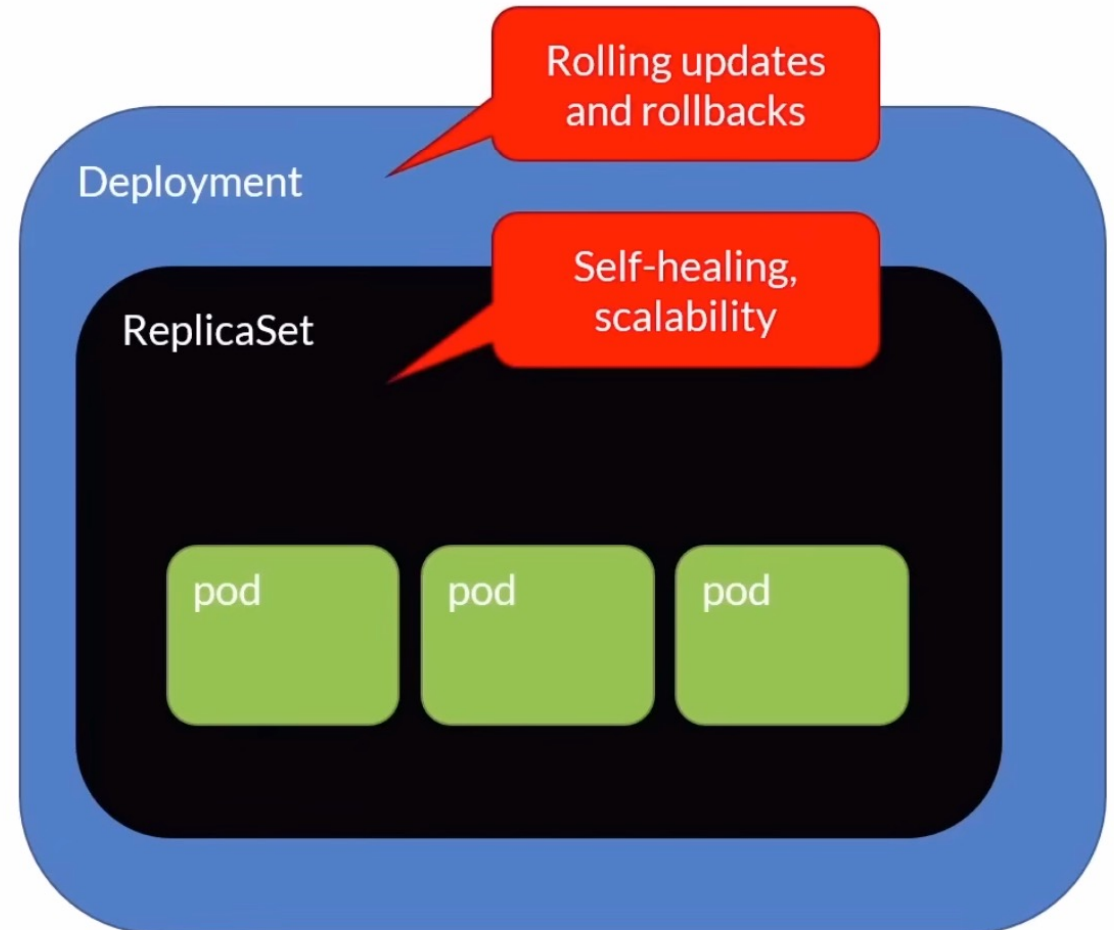
Command	Brief Description
<code>kubectl apply -f [replicaset-definition.yaml]</code>	Create a ReplicaSet
<code>kubectl get rs</code>	List all ReplicaSets
<code>kubectl describe rs [rsName]</code>	Show ReplicaSet info
<code>kubectl delete -f [replicaset-definition.yaml]</code>	Delete a ReplicaSet
<code>kubectl delete rs [rsName]</code>	Delete a ReplicaSet Using it's name

Workloads - Pods vs Deployment

- Pods don't
 - Self-heal
 - Scale
 - Updates
 - Rollback
- Deployments can!

Workloads - Deployments

- A deployment manage a single pod template
- You create a deployment for each microservice
 - front-end
 - back-end
 - Image-processor
 - Creditcard-processor
- Deployments create ReplicaSets in the background
- Don't interact with the ReplicaSets directly



Workloads - Deployments

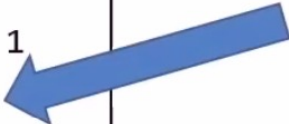
- Replicas
 - Number of pod instances
- revisionHistoryLimit
 - Number of previous iterations to keep
- Strategy
 - RollingUpdate
 - Cycle through updating pods
 - Recreate
 - All existing pods are killed before new ones are created

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: deploy-example
spec:
  replicas: 3
  revisionHistoryLimit: 3
  selector:
    matchLabels:
      app: nginx
      env: prod
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1
  template:
    <pod template>
```

Workloads - Deployments

Deployment definition

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: deploy-example
spec:
  replicas: 3
  revisionHistoryLimit: 3
  selector:
    matchLabels:
      app: nginx
      env: prod
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1
  template:
    <pod template>
```



+

Pod definition

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: nginx
    type: prod
spec:
  containers:
    - name: nginx
      image: nginx:stable-alpine
      ports:
        - containerPort: 80
```

=

Final Deployment definition

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: deploy-example
spec:
  replicas: 3
  revisionHistoryLimit: 3
  selector:
    matchLabels:
      app: nginx
      env: prod
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1
  template:
    metadata:
      labels:
        app: nginx
        type: prod
    spec:
      containers:
        - name: nginx
          image: nginx:stable-alpine
          ports:
            - containerPort: 80
```

kubectl – Deployment Cheat Sheet

Command	Brief Description
<code>kubectl create deploy [deploymentName] --image=busybox --replicas=3 --port=80</code>	The imperative way
<code>Kubectl apply -f [deploy-definition.yaml]</code>	Create a deployment
<code>kubectl get deploy</code>	List deployments
<code>kubectl describe deploy [deploymentName]</code>	Get info
<code>kubectl get rs</code>	List ReplicaSets
<code>kubectl delete -f [deploy-definition.yaml]</code>	Delete a deployment
<code>kubectl delete deploy [deploymentName]</code>	Delete a deployment using its name

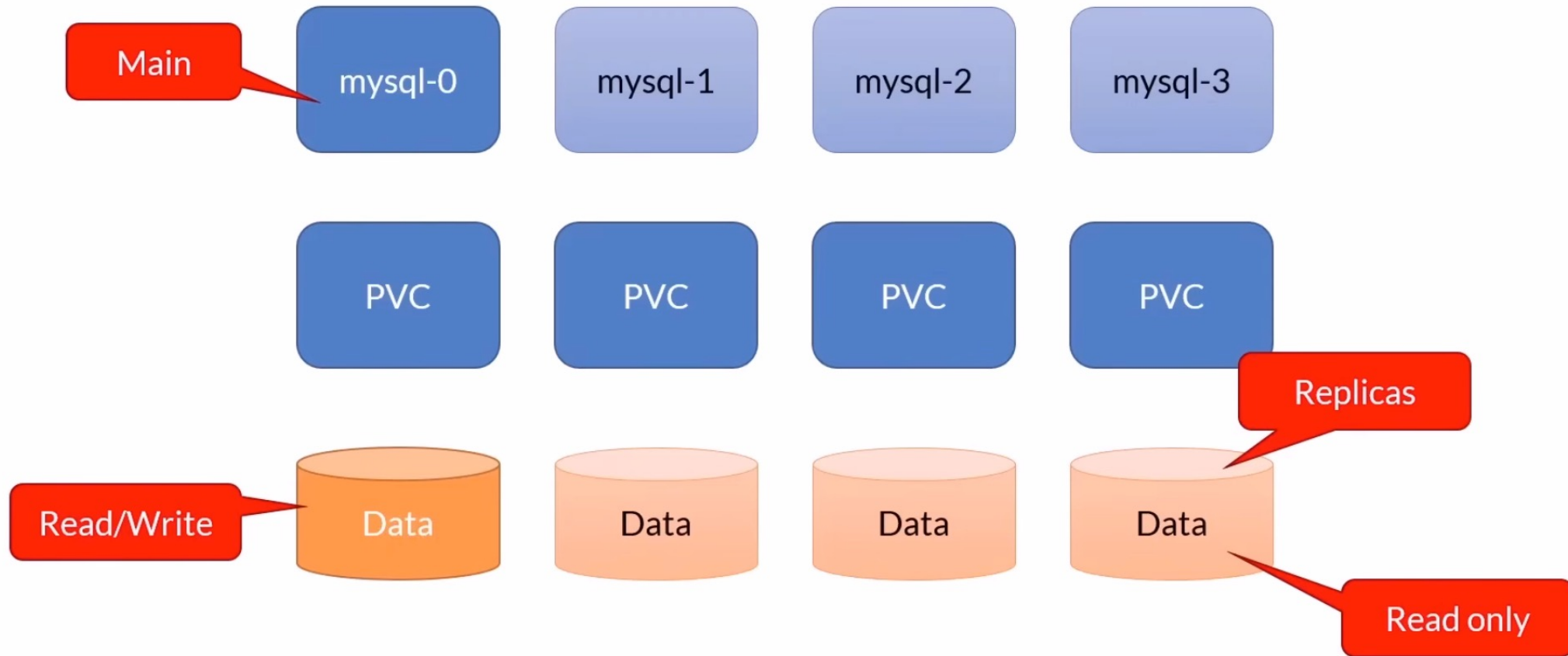
Workloads - DaemonSet

- Ensures all Nodes(or a subset) run an instance of a Pod
- Scheduled by the scheduler controller and run by the daemon controller
- As nodes are added to the cluster, Pods are added to them
- Typical uses
 - Running a cluster storage daemon
 - Running a logs collection daemon on every node
 - Running a node monitoring daemon on every node

kubectl – DaemonSets Cheat Sheet

Command	Brief Description
<code>kubectl apply -f [daemonset-definition.yaml]</code>	Create a DaemonSet
<code>kubectl get ds</code>	List all DaemonSet
<code>kubectl describe ds [dsName]</code>	Show DaemonSet info
<code>kubectl delete -f [daemonset-definition.yaml]</code>	Delete a DaemonSet
<code>kubectl delete ds [dsName]</code>	Delete a DaemonSet Using it's name

Workloads - Scaling a database



Workloads - StatefulSet

- For Pods that must persist or maintain state
- Unlike a Deployment, a StatefulSet maintains a sticky identity for each of their Pods
- Each has a persistent identifier (name-x)
- If a pod dies, it is replaced with another one using the identifier
- Creates a series of pods in sequence from 0 to X and deletes them from X to 0
- Typical uses
 - Stable, unique network identifiers
 - Stable, databases using persistent storage

Workloads – StatefulSet YAML

```
apiVersion: v1
kind: Service
metadata:
  name: mysql
  labels:
    app: mysql
spec:
  ports:
    - name: mysql
      port: 3306
  clusterIP: None
  selector:
    app: mysql
```

Headless service

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
spec:
  serviceName: mysql
  replicas: 4
  selector:
    matchLabels:
      run: nginx-sts-demo
  template:
    metadata:
      labels:
        run: nginx-sts-demo
    spec:
      containers:
        - name: init-mysql
          image: mysql:5.7
          volumeMounts:
            - name: conf
              mountPath: /mnt/conf.d
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        storageClassName: default
        accessModes:
          - ReadWriteOnce
        resources:
          requests:
            storage: 1Gi
```

Cloud Storage

Workloads - StatefulSet

- Containers are stateless by design
- StatefulSets offers a solution for stateful scenarios
- A better approach could be to use the Cloud provider database services
- Deleting a StatefulSet will not delete the PVCs
 - You have to do this manually

kubectl – StatefulSets Cheat Sheet

Command	Brief Description
<code>kubectl apply -f [statefulset-definition.yaml]</code>	Create a StatefulSet
<code>kubectl get sts</code>	List StatefulSets
<code>kubectl describe sts [stsName]</code>	Show StatefulSet info
<code>kubectl delete -f [statefulset-definition.yaml]</code>	Delete a StatefulSet
<code>kubectl delete sts [stsName]</code>	Delete a StatefulSet Using it's name

Workloads - Job

- Workload for short lived tasks
- Creates one or more pods and ensures that a specified number of them successfully terminate
- As pods successfully complete, the job tracks the successful completions
- When a specified number of successful completions is reached, the job completes
- By default, jobs with more than 1 pod will create them one after the other. To create them at the same time, add parallelism.

Workloads - Job YAML definition

How many pods
should run in parallel

```
apiVersion: batch/v1
kind: Job
metadata:
  name: pi
spec:
  activeDeadlineSeconds 30
  parallelism: 3
  completions: 3
  template:
    spec:
      containers:
      - name: pi
        image: perl
        command: ["perl", "-Mbignum=bpi", "-wle", "print bpi(2000)"]
      restartPolicy: Never
```

Max seconds to run

how many successful pod
completions are needed to mark a
job completed

Default is Always

kubectl – Job Cheat Sheet

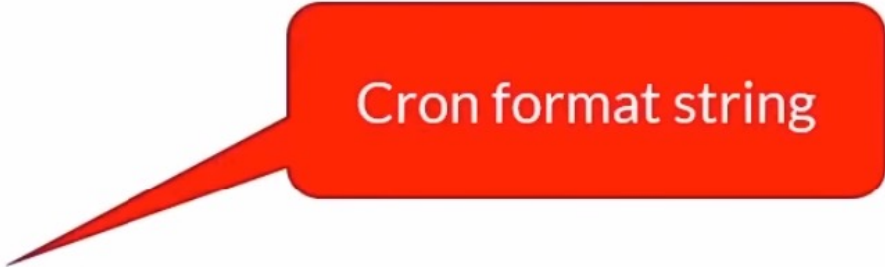
Command	Brief Description
<code>kubectl create job [jobName] --image=busybox</code>	The imperative way
<code>kubectl apply -f [job-definition.yaml]</code>	Create a Job
<code>kubectl get job</code>	List Jobs
<code>kubectl describe job [jobName]</code>	Show Job info
<code>kubectl delete -f [job-definition.yaml]</code>	Delete a Job
<code>kubectl delete job [jobName]</code>	Delete a Job Using it's name

Workloads - CronJob

- An extension of the job
- Provides a method of executing jobs on a cron-like schedule
- UTC only

CronJob Definition YAML

```
apiVersion: batch/v1beta1
kind: CronJob
metadata:
  name: hello-cron
spec:
  schedule: "* * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: busybox
              image: busybox
              command: ["echo", "Hello from the CronJob"]
          restartPolicy: Never
```



Cron format string

CronJob Troubleshooting

- History
 - The last 3 successful jobs are kept
 - The last failed job is kept
 - Setting `successfulJobsHistoryLimit` to zero keeps none

```
kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
myjob-1556555640-9bc5r	0/1	Completed	0	3m6s
myjob-1556555700-cm6wk	0/1	Completed	0	2m6s
myjob-1556555760-62wf5	0/1	Completed	0	66s

kubectl – Job Cheat Sheet

Command	Brief Description
<code>kubectl create cronjob [jobName] --image=busybox --schedule="*/1 * * * *" -- bin/sh -c "date;"</code>	The imperative way
<code>kubectl apply -f [cronjob-definition.yaml]</code>	Create a CronJob
<code>kubectl get cj</code>	List CronJobs
<code>kubectl describe cj [cronjobName]</code>	Show CronJob info
<code>kubectl delete -f [cronjob-definition.yaml]</code>	Delete a CornJob
<code>kubectl delete cj [cronjobName]</code>	Delete a CornJob Using the it's name

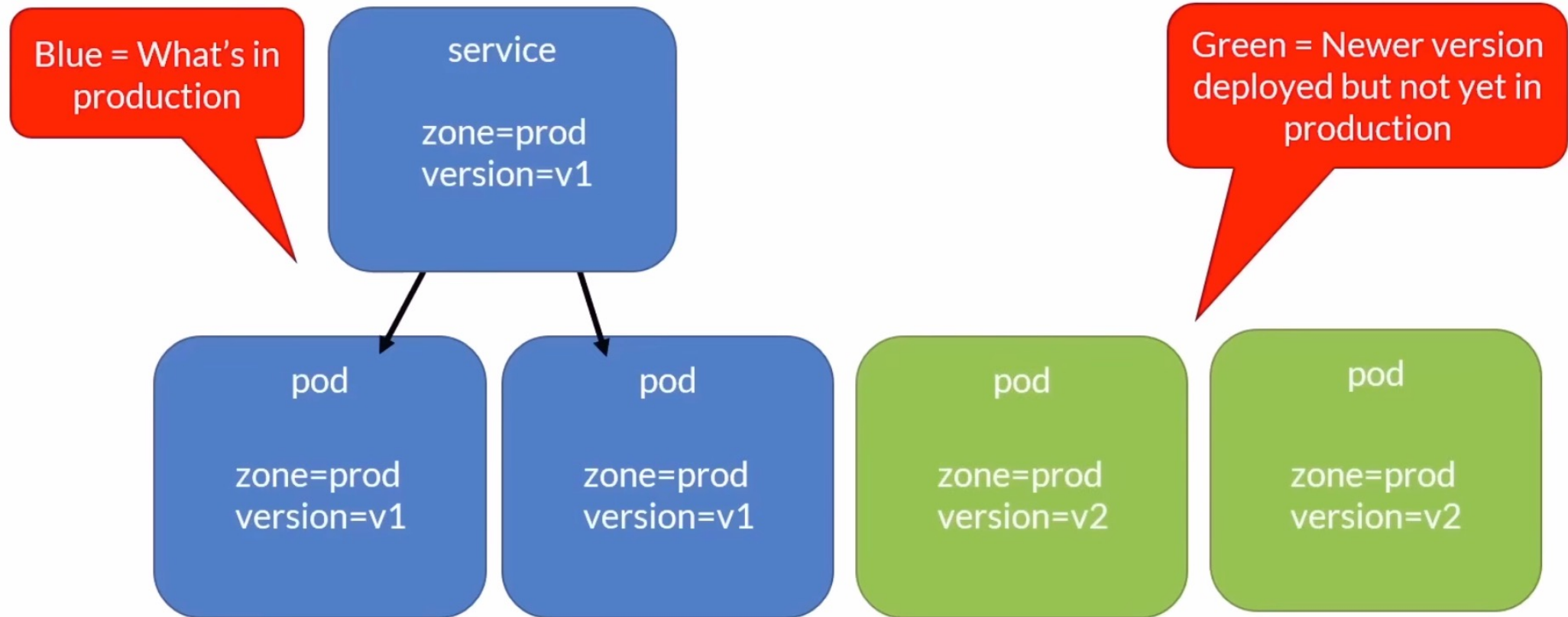
Workloads - RollingUpdate

- maxSurge
 - Maximum number of Pods that can be created over the desired number of pods
 - Value or percentage
- maxUnavailable
 - Maximum number of Pods that can be unavailable during the update process
- Default strategy with maxSurge and maxUnavailable both set to 25%

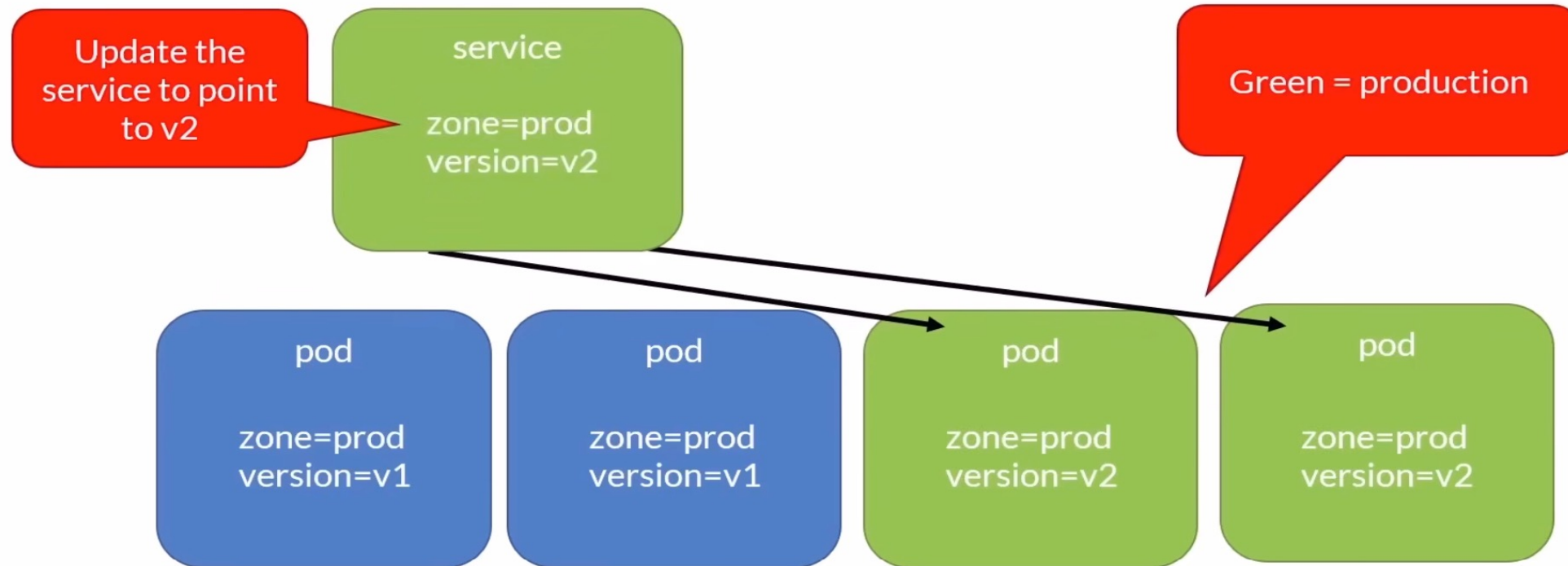
kubectl – RollingUpdates Cheat Sheet

Command	Brief Description
<code>kubectl apply -f [rollingupdate-definition.yaml]</code>	Update a deployment
<code>kubectl rollout status</code>	Get the progress of the update
<code>kubectl rollout history deployment [deploymentName]</code>	Get the history of the deployment
<code>kubectl rollout undo [deploymentName]</code>	Rollback a deployment
<code>kubectl rollout undo [deploymentName] --to-revision=[revision#]</code>	Rollback to a revision number

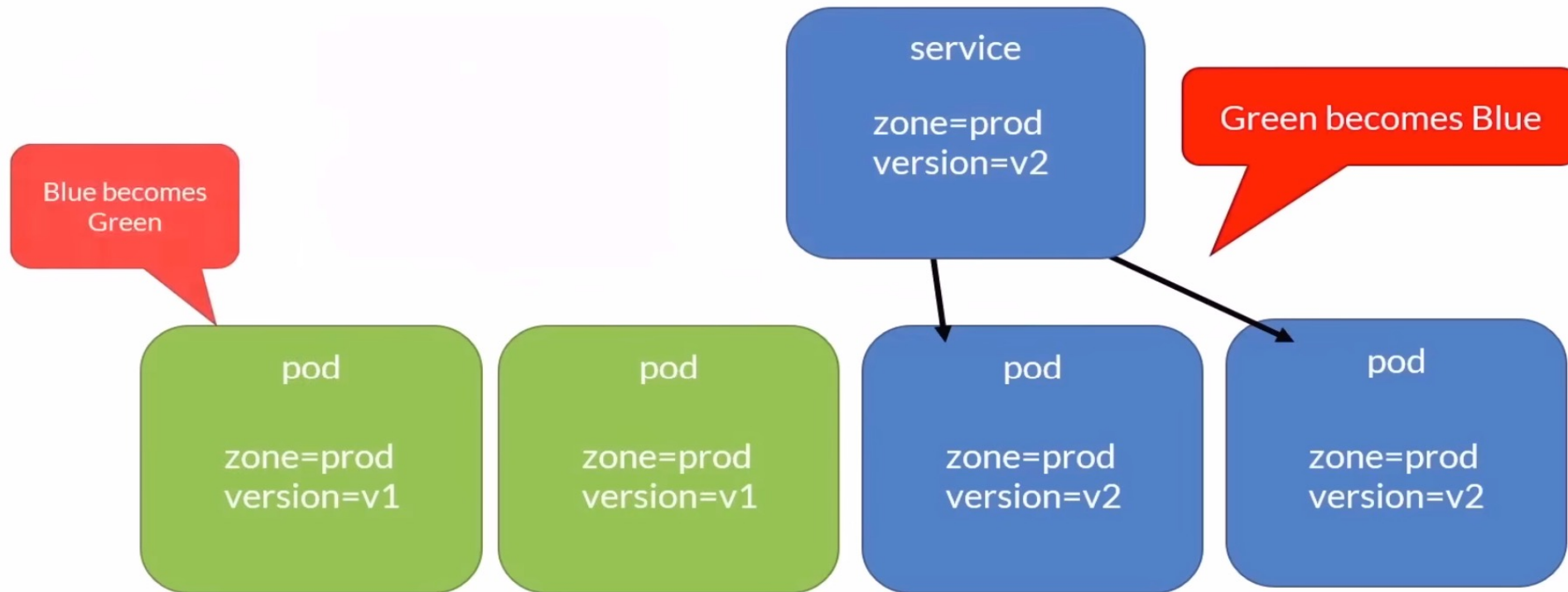
Workloads - Blue-Green Deployments



Workloads - Blue-Green Deployments



Workloads - Blue-Green Deployments



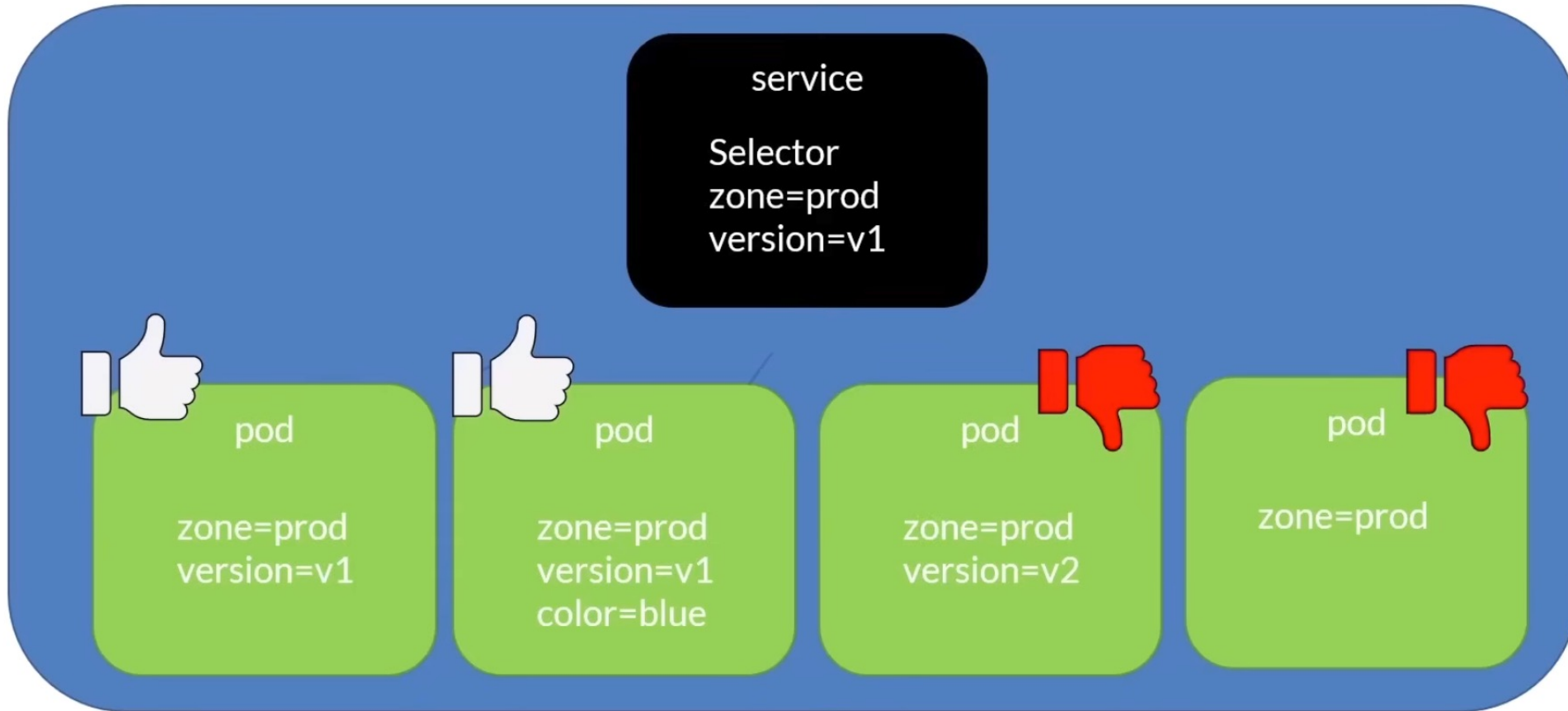
Workloads - Blue-Green Deployments

- Does not solve the new database schema problem entirely
- You need to over provision the cluster size

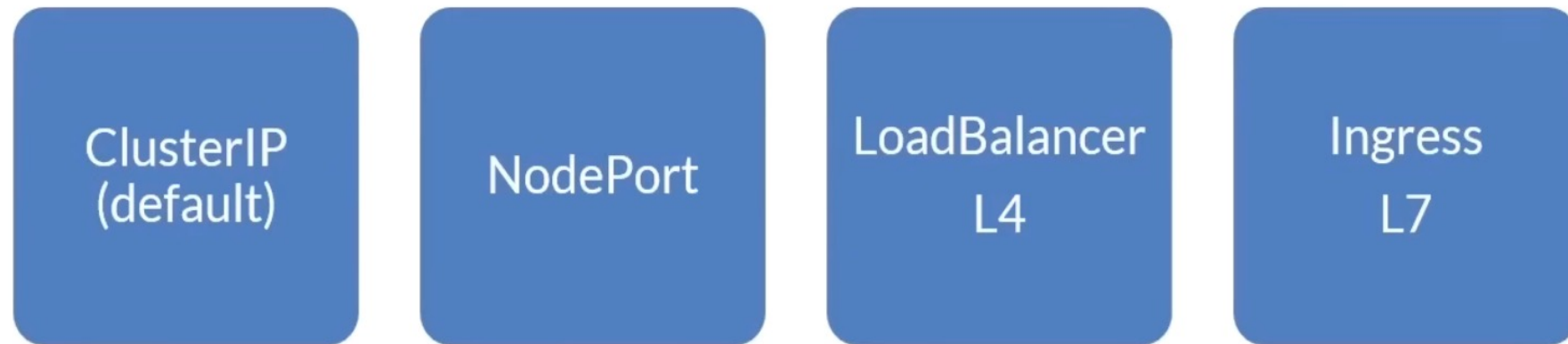
Services

- A service is another type of k8s object
- Pods Ips are unreliable but service Ips are durable (unlike pods)
 - Static IP address
 - Static DNS name
 - [serviceName].[namespace].svc.cluster.local
- Services are ways to access pods
- Target pods using selectors

Label Selectors



Services



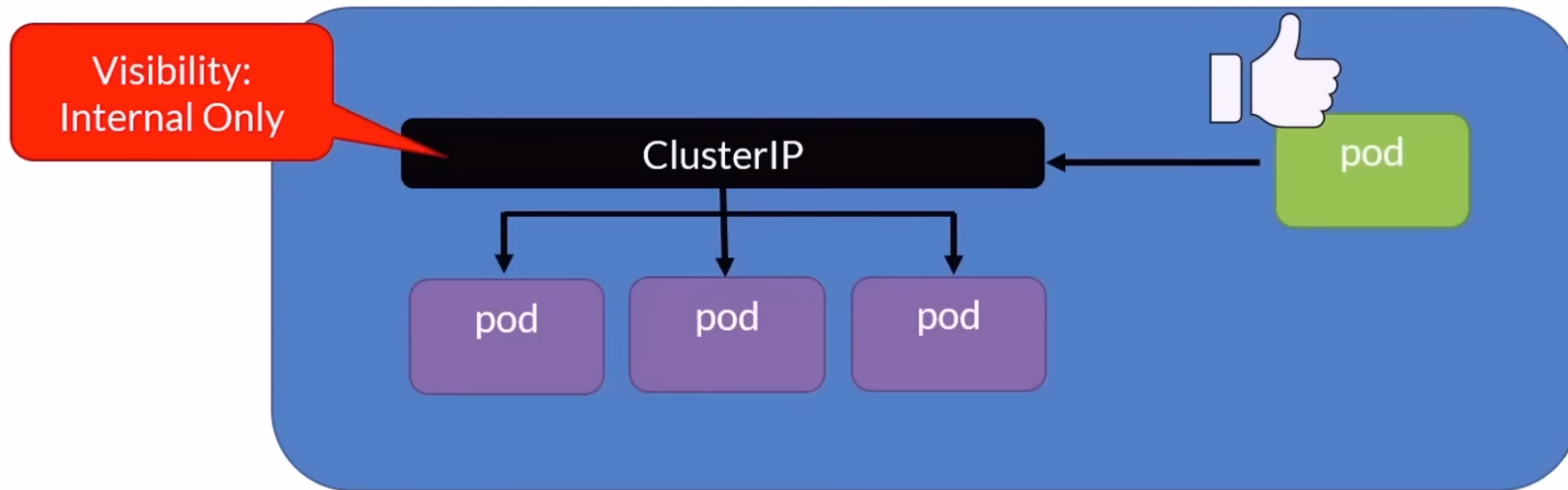
Services – Cluster IP

- ClusterIP is the default service
- Visibility
 - Cluster internal
- Port
 - The port service is listening to
- TargetPort
 - Redirecting to the port the pod(s) are listening to
- Load balanced using round robin
 - Session affinity is configurable
- When to use
 - To provide a durable way to communicate with pods inside the cluster

Service Definition

```
apiVersion: v1
kind: Service
metadata:
  name: svc-example
spec:
  ports:
    - port: 80
      targetPort: 8080
  selector:
    app: app-example
    env: prod
```

Services - ClusterIP



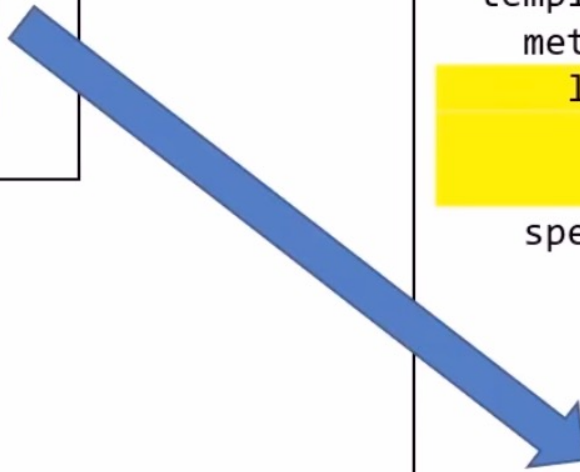
Services – ClusterIP definition

Service definition

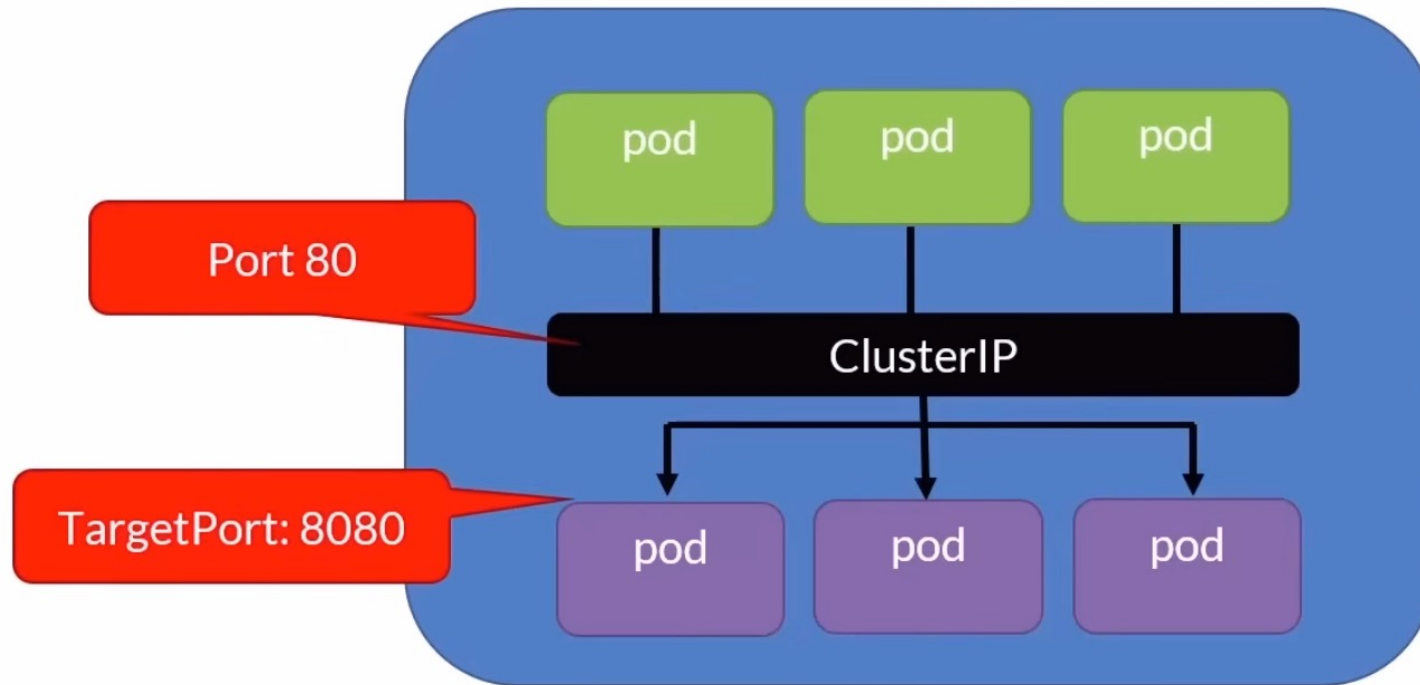
```
apiVersion: v1
kind: Service
metadata:
  name: svc-example
spec:
  ports:
    - port: 80
      targetPort: 8080
  selector:
    app: app-example
    env: prod
```

Deployment definition

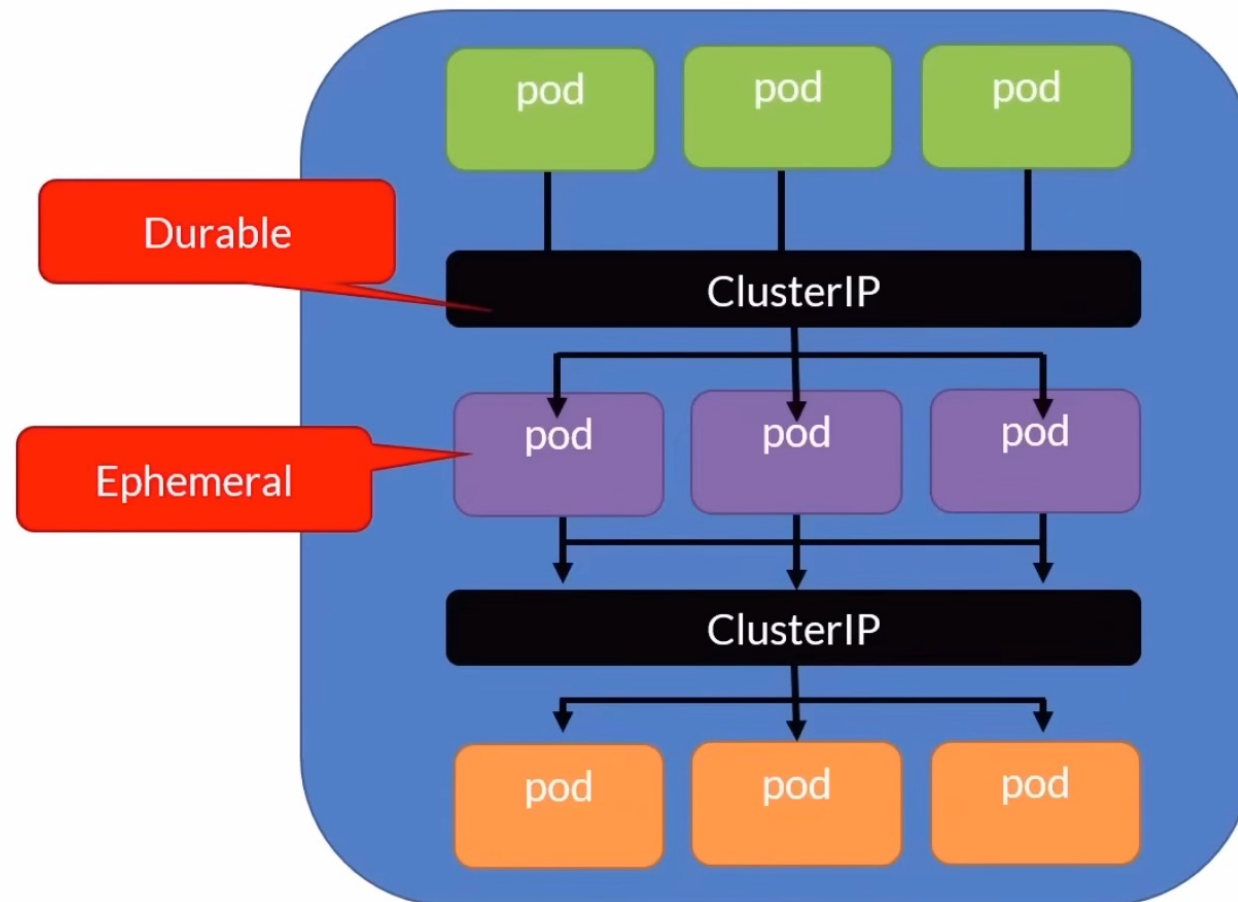
```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: deploy-example
spec:
  replicas: 3
  revisionHistoryLimit: 3
  selector:
    matchLabels:
      app: app-example
      env: prod
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1
  template:
    metadata:
      labels:
        app: app-example
        env: prod
    spec:
      containers:
        - name: nginx
          image: nginx:stable-alpine
          ports:
            - containerPort: 8080
```



ClusterIP - Accessing pods



ClusterIP – Accessing Microservices



ClusterIP – Cheat Sheet

Command	Brief Description
<code>kubectl expose po [podName] --port=80 --target-port=8080 --name=frontend</code>	Create a service to expose a pod
<code>kubectl expose deploy [deployName] --port=80 --target-port=8080</code>	Create a service to expose a deployment
<code>Kubectl apply -f [svc-definition.yaml]</code>	Deploy the service
<code>kubectl get svc</code>	List the services list
<code>kubectl get svc -o wide</code>	Get extra Info
<code>kubectl describe svc [serviceName]</code>	Describe the service
<code>kubectl delete -f [svc-definition.yaml]</code>	Delete the service
<code>kubectl delete svc [serviceName]</code>	Delete the svc using its name

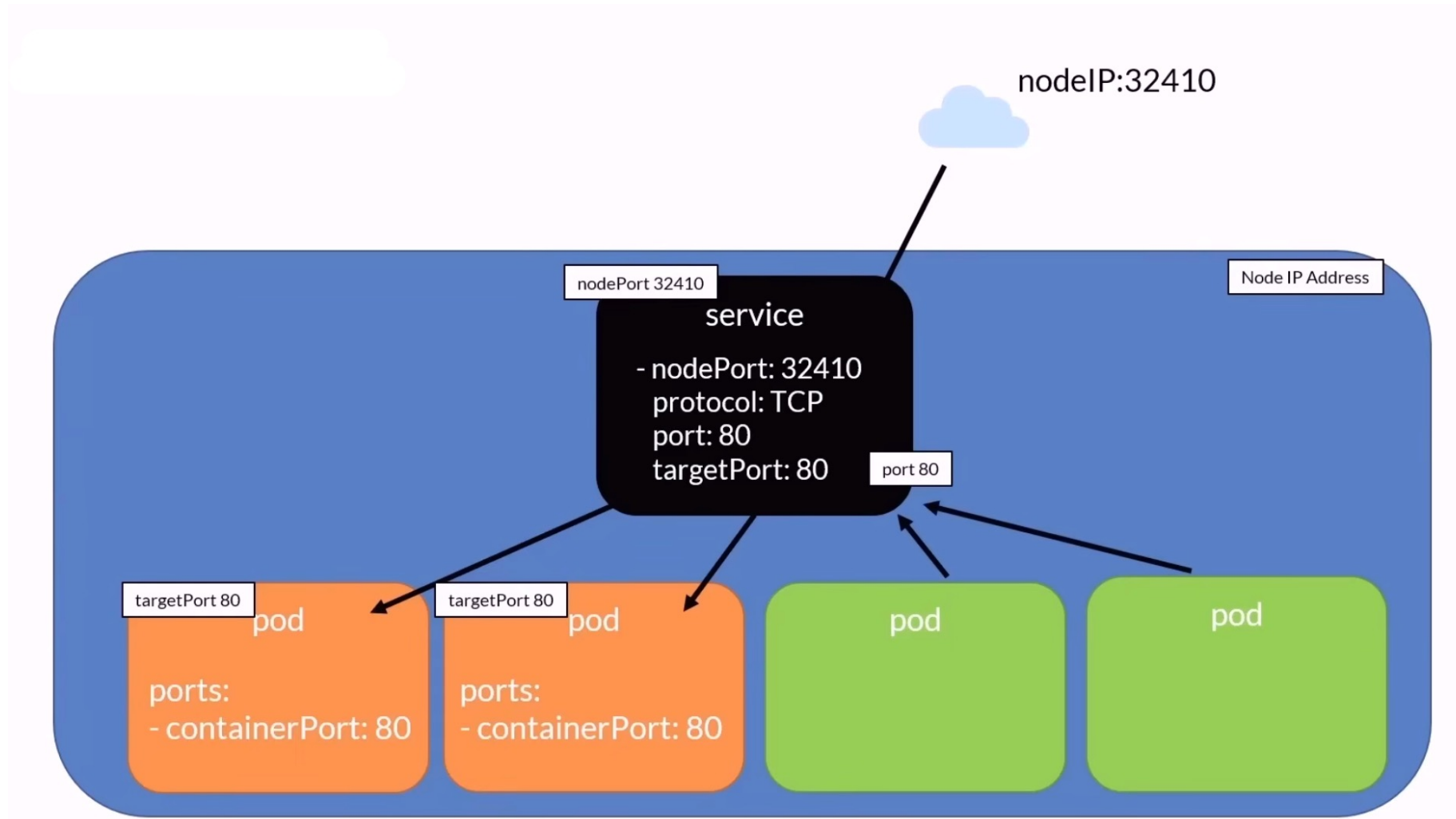
Services – NodePort

- NodePort extend the ClusterIP service
- Visibility
 - Internal and external
- NodePort
 - The port the service is listening to externally
 - Statically defined, or dynamically taken from a range between 30000-32767
- Port
 - The port the service is listening to internally
- TargetPort
 - Redirecting to the port the pod(s) are listening to
- Nodes
 - Must have public IP address
- Use any Node IP + nodePort to access the service

Service Definition

```
apiVersion: v1
kind: Service
metadata:
  name: svc-example
spec:
  type: NodePort
  ports:
    - nodePort: 32410
      protocol: TCP
      port: 80
      targetPort: 80
  selector:
    app: nginx
    env: prod
```

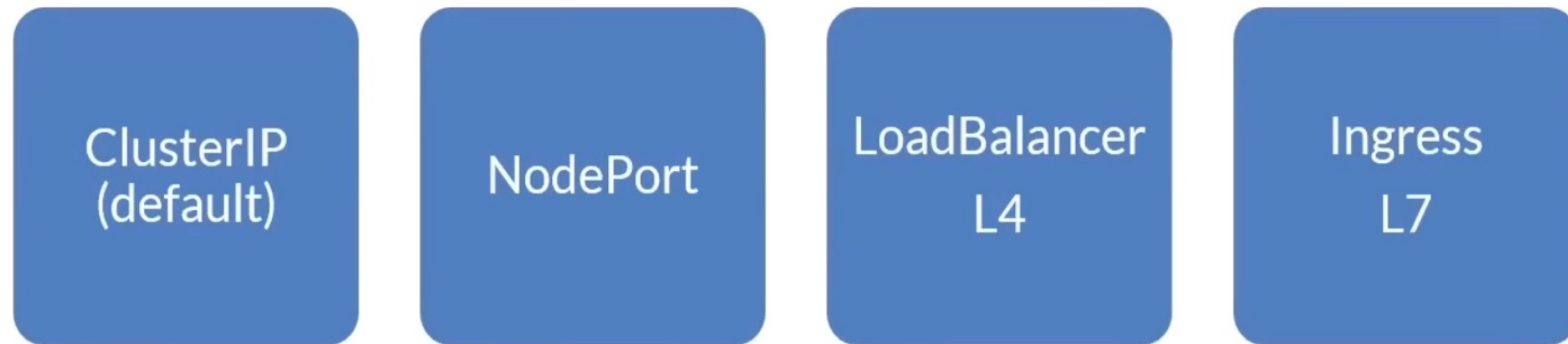
Services - NodePort



NodePort – Cheat Sheet

Command	Brief Description
<code>kubectl expose po [podName] --port=80 --target-port=8080 --type=NodePort</code>	Create a service to expose a pod
<code>kubectl expose deploy [deployName] --port=80 --target-port=8080 --type=NodePort --name=frontend</code>	Create a service to expose a deployment
<code>Kubectl apply -f [nodeport-definition.yaml]</code>	Deploy the service
<code>kubectl get svc</code>	List the services list
<code>kubectl get svc -o wide</code>	Get extra Info
<code>kubectl describe svc [serviceName]</code>	Describe the service
<code>kubectl delete -f [nodeport-definition.yaml]</code>	Delete the service
<code>kubectl delete svc [serviceName]</code>	Delete the svc using it's name

Services



Services – Load Balancing

- Layer 4 Load balancing (Load Balancer L4)
 - Operating at the transport level TCP
 - Unable to make decisions based on content
 - Simple algorithms such as round-robin routing
- Layer 7 Load balancing (Ingress L7)
 - Operates at the application level (HTTP, SMTP, SMSC etc.)
 - Able to make decisions based on the actual content of each message
 - More intelligent load balancing decisions and content optimizations
 - Routing rules

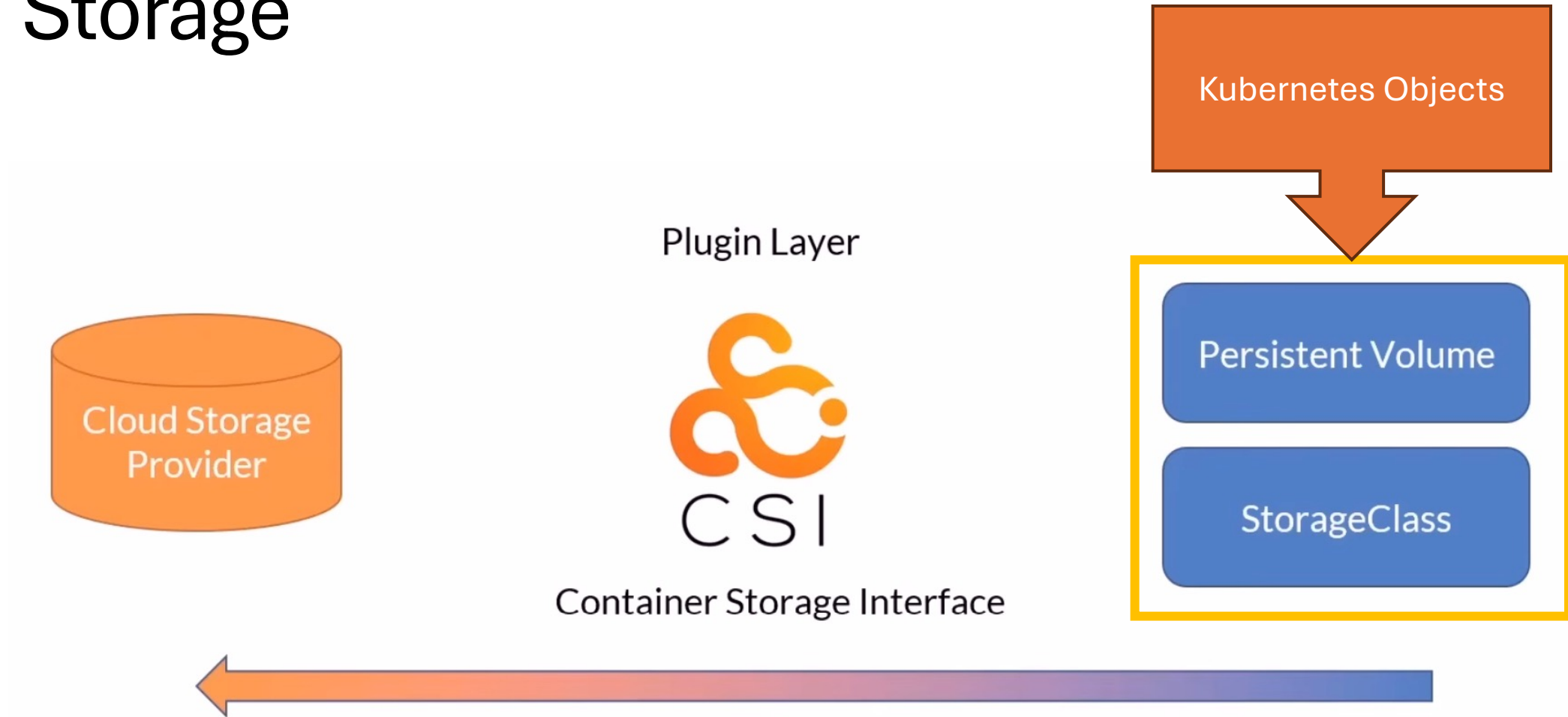
Container are ephemeral and stateless

- You usually don't store data in containers
- Non-persistent data
 - Locally on a writeable layer
 - It's the default, just write to the file system
 - When containers are destroyed, so the data inside them

Storage & Persistence – Volumes

- We need to store data outside the container in volume
- Volume let containers store data into external storage systems
- Vendors create plugins for their storage systems according to the Container Storage Interface
- Two ways to create storage
 - Static and dynamic

Storage



Storage – The Static Way

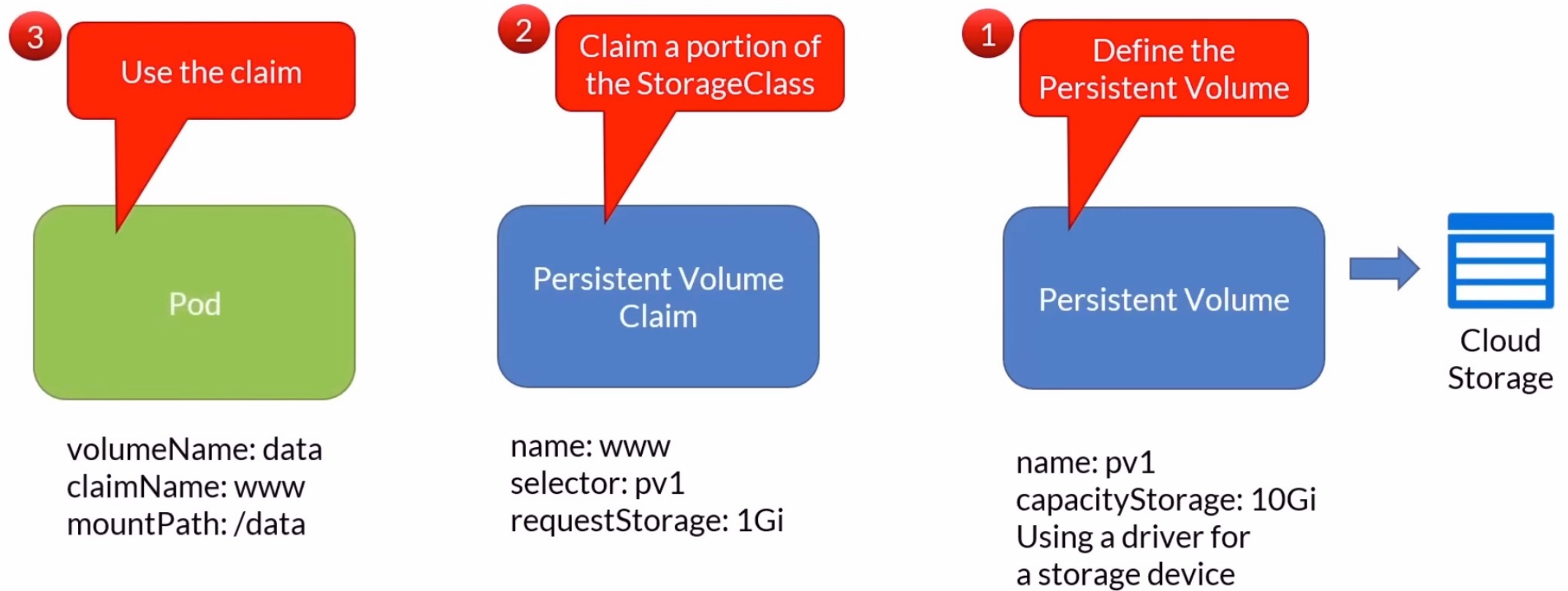
Persistent Volumes and Claims

- Persistent Volumes
 - Represent a storage resource
 - Cluster wide
 - Provisioned by an administrator
- Persistent Volume Claim
 - A one to one mapping to a persistent volume
- One or more pods can use a Persistent Volume Claim
- Can be consumed by any of the containers within the pod

Types of Persistent Volumes

- GCEPersistentDisk
- AWSElasticBlockStore
- AzureFile
- AzureDisk
- CSI
- FC(Fibre Channel)
- FlexVolume
- Flocker
- NFS
- ISCSI
- RBD(Ceph Block Device)
- CephFS
- Cinder(OpenStack block storage)
- Glusterfs
- VsphereVolume
- Quobyte Volumes
- Portworx Volumes
- ScaleIO Volumes
- StorageOS
- HostPath
 - Single Node Testing only – local storage is not supported in any way and **WILL NOT WORK** in a multi-node cluster

Persistent Volumes and Claims



Reclaim Policies

- Delete
 - Delete data upon pods deletion
 - The default
- Retain
 - Keeps the data upon pods deletion

Note: The default is deletion of data upon pod deletion***

Access Modes

- ReadWriteMany
 - The volume can be mounted as read-write by many pods
- ReadOnlyMany
 - The volume can be mounted read-only by many pods
- ReadWriteOnce
 - The volume can be mounted as read-write by a single pod
 - The other pods are in read-only mode
 - The one that has mounted the volume first will be able to write

Persistent Volume YAML

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv001
  labels:
    type: local
spec:
  storageClassName: ssd
  capacity:
    storage: 1Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  hostPath:
    path: "/data/"
```

hostPath is a plugin
allowing local storage
on the node



hostPath works only on
single-node clusters

Persistent Volume Claim

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv001
  labels:
    type: local
spec:
  capacity:
    storage: 10Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  hostPath:
    path: "/data/"
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 8Gi
  selector:
    matchLabels:
      type: local
```

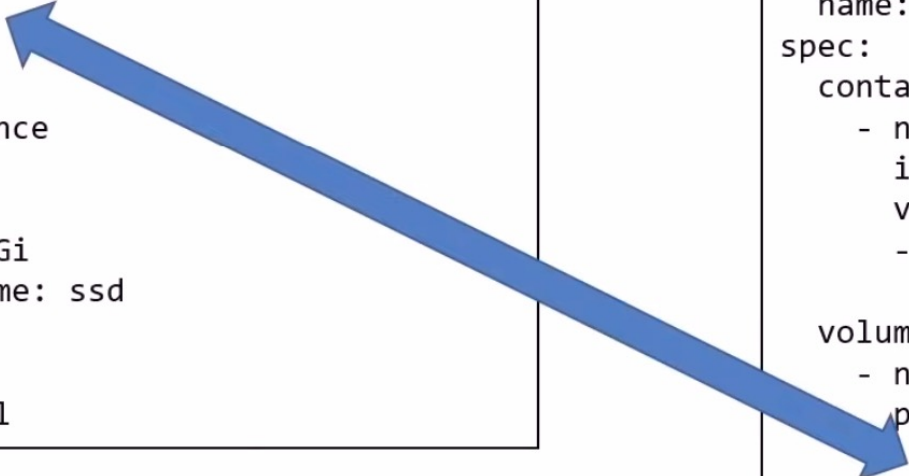
Access modes must match

Claims 8Gi on the Persistent Volume

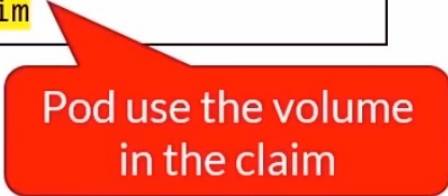
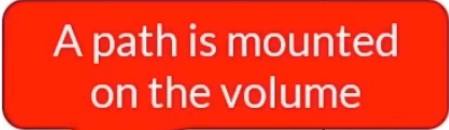
No one can claim the remaining 2 Gi

Persistent Volume Claim

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 8Gi
  storageClassName: ssd
  selector:
    matchLabels:
      type: local
```



```
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
    - name: myfrontend
      image: nginx
      volumeMounts:
        - mountPath: "/var/www/html"
          name: mypd
  volumes:
    - name: mypd
      persistentVolumeClaim:
        claimName: myclaim
```



A path is mounted
on the volume

Pod use the volume
in the claim

Persistent Volume States

- Available
 - A free resource that is not yet bound to a claim
- Bound
 - The volume is bound to be a claim
- Released
 - The claim has been deleted, but the resource is not yet reclaimed by the cluster
- Failed
 - The volume has failed its automatic reclamation

kubectl – PV & PVC Cheat Sheet

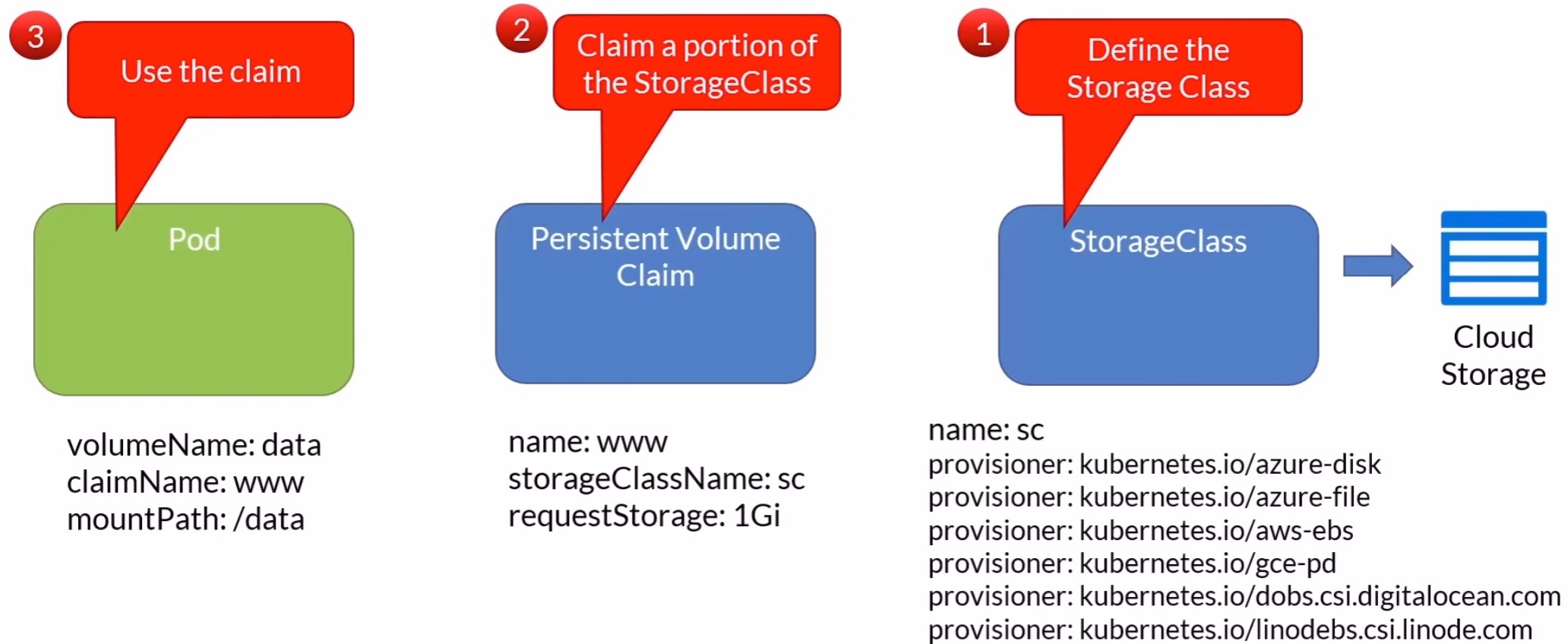
Command	Brief Description
<code>kubectl apply -f [definition.yaml]</code>	Deploy the PVs and PVCs
<code>kubectl get pv</code>	List the PV list
<code>kubectl get pvc</code>	List the PVC list
<code>kubectl describe pv [pvName]</code>	Describe the PV
<code>kubectl describe pvc [pvcName]</code>	Describe the PVC
<code>kubectl delete -f [definition.yaml]</code>	Delete the PVs and PVCs
<code>kubectl delete pv [pvName]</code>	Delete the pv using it's name
<code>kubectl delete pvc [pvcName]</code>	Delete the pvc using it's name

Storage – The Dynamic Way

StorageClass

- Describe the “classes” of storage offered by the admin
- An abstraction on top of an external storage resource
- No need to set capacity
- Eliminates the need for the admin to pre-provision a persistent volume

Persistent Volumes and Claims Using StorageClass



StorageClass – Reclaim Policies

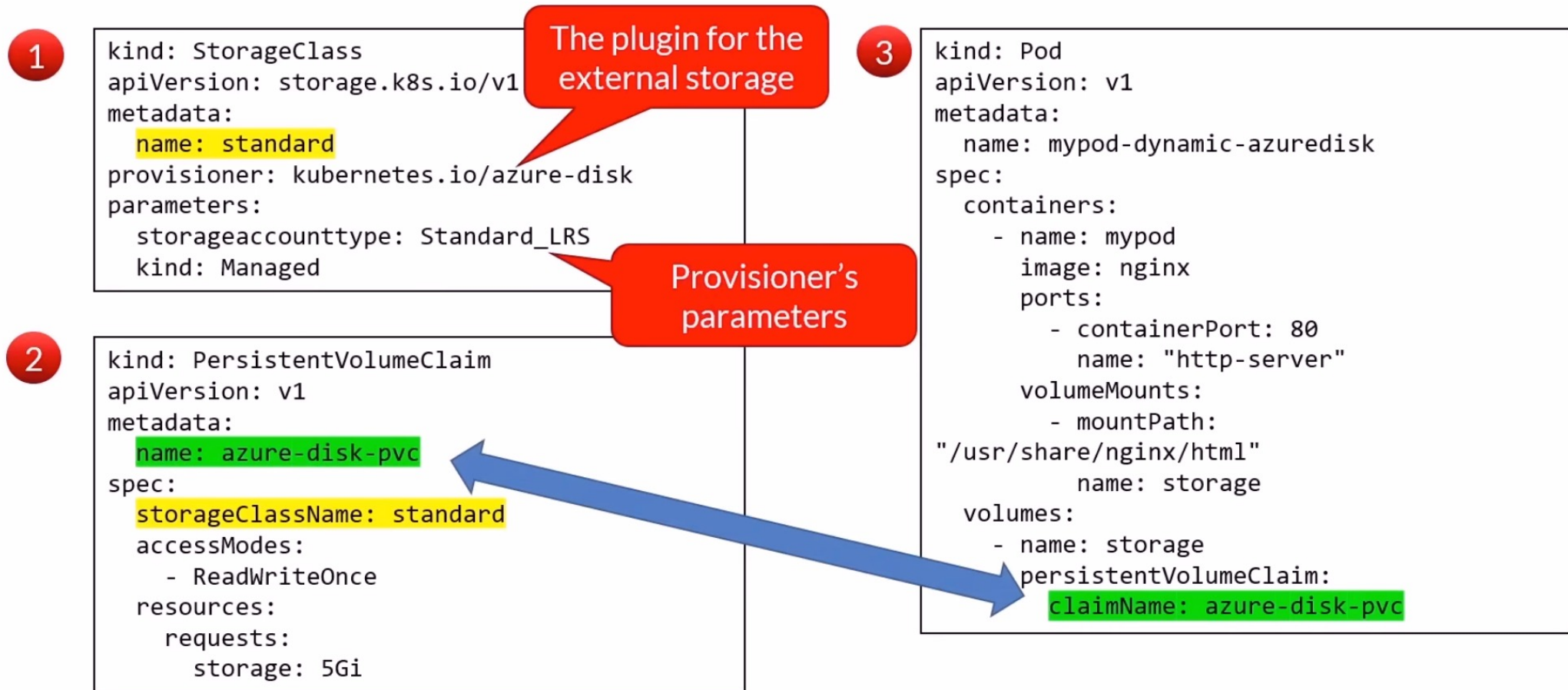
- Delete
 - Delete data upon pods deletion
 - The default
- Retain
 - Keeps the data upon pods deletion

Note: The default is deletion of data upon pod deletion***

StorageClass - Access Modes

- ReadWriteMany
 - The volume can be mounted as read-write by many pods
- ReadOnlyMany
 - The volume can be mounted read-only by many pods
- ReadWriteOnce
 - The volume can be mounted as read-write by a single pod
 - The other pods are in read-only mode
 - The one that has mounted the volume first will be able to write

StorageClass YAML



PersistentVolume vs StorageClass

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv001
  labels:
    type: local
spec:
  storageClassName: ssd
  capacity:
    storage: 1Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Recycle
  hostPath:
    path: "/data/"
```

Capacity defined

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: standard
provisioner: kubernetes.io/azure-disk
parameters:
  storageaccounttype: Standard_LRS
  kind: Managed
```

No capacity is specified

kubectl – StorageClass Cheat Sheet

Command	Brief Description
<code>kubectl apply -f [definition.yaml]</code>	Deploy the StorageClass or PVCs
<code>kubectl get sc</code>	List the StorageClass list
<code>kubectl get pvc</code>	List the PVC list
<code>kubectl describe sc [className]</code>	Describe the StorageClass
<code>kubectl delete -f [definition.yaml]</code>	Delete the SC and PVC
<code>kubectl delete sc [className]</code>	Delete the SC using it's name

ConfigMaps

- Decouple and externalize configuration
- Referenced as environment variables
- Created from
 - Manifest
 - Files
 - Directories (Containing one or more file)
- Static meaning that if you change values, the containers will have to be restarted to get them

ConfigMaps YAML

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: cm-example
data:
  state: Michigan
  city: Ann Arbor
  content: |
    Look at this,
    its multiline!
```

Values

Use a pipe for
multi-lines values

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-example
spec:
  containers:
    - name: test-container
      image: busybox
      env:
        - name: STATE
          valueFrom:
            configMapKeyRef:
              name: cm-example
              key: state
```

ConfigMaps YAML

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: cm-example
data:
  state: Michigan
  city: Ann Arbor
  content: |
    Look at this,
    its multiline!
```

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-example
spec:
  containers:
    - name: test-container
      image: busybox
      env:
        - name: STATE
          valueFrom:
            configMapKeyRef:
              name: cm-example
              key: state
```

Get the value from
the configMap

ConfigMaps Values are static



ConfigMaps and Volumes

- Solve the static issue
- Updates to values are reflected in containers
- Each Key/Value pair is seen as a file in the mounted directory



ConfigMaps and Volumes

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: cm-example
data:
  state: Michigan
  city: Ann Arbor
  content: |
    Look at this,
    its multiline!
```

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-example
spec:
  volumes:
    - name: volmap
      configMap:
        name: cm-example
  containers:
    - name: test-container
      image: nginx
      volumeMounts:
        - name: volmap
          mountPath: /etc/config
```

Get the value from
the configMap

Values will be found
as files in that folder

kubectl – ConfigMaps Cheat Sheet

Command	Brief Description
<code>kubectl create configmap literal-example --from-literal="city=Ann Arbor" --from-literal=state=Michigan</code>	The Imperative Way Create ConfigMaps
<code>kubectl apply -f [cf.yaml]</code>	The declarative way
<code>kubectl create cm [name] --from-file=myconfig.txt</code>	From a file
<code>kubectl create cm [name] --from-file=config/</code>	From a folder
<code>kubectl get cm</code>	List the ConfigMaps
<code>kubectl describe configmap [name]</code>	Get ConfigMap details by its name
<code>kubectl get cm [name] -o YAML</code>	Save a ConfigMap in a YAML file
<code>kubectl delete -f [cf.yaml]</code>	Delete a ConfigMap

Secrets



Builtin Type	Usage
Opaque	arbitrary user-defined data
kubernetes.io/service-account-token	service account token
kubernetes.io/dockercfg	serialized ~/.dockercfg file
kubernetes.io/dockerconfigjson	serialized ~/.docker/config.json file
kubernetes.io/basic-auth	credentials for basic authentication
kubernetes.io/ssh-auth	credentials for SSH authentication
kubernetes.io/tls	data for a TLS client or server
bootstrap.kubernetes.io/token	bootstrap token data

Secrets

Builtin Type	
Opaque	arbitrary user-defined data
kubernetes.io/service-account-token	service account token
kubernetes.io/dockercfg	serialized ~/.dockercfg file
kubernetes.io/dockerconfigjson	serialized ~/.docker/config.json file
kubernetes.io/basic-auth	credentials for basic authentication
kubernetes.io/ssh-auth	credentials for SSH authentication
kubernetes.io/tls	data for a TLS client or server
bootstrap.kubernetes.io/token	bootstrap token data

Private registry
credentials

Secrets

Builtin Type	Usage
Opaque	Arbitrary user-defined data
kubernetes.io/service-account-token	Service account token
kubernetes.io/dockercfg	Serialized ~/.dockercfg file
kubernetes.io/dockerconfigjson	Serialized ~/.docker/config.json file
kubernetes.io/basic-auth	Credentials for basic authentication
kubernetes.io/ssh-auth	Credentials for SSH authentication
kubernetes.io/tls	Data for a TLS client or server
bootstrap.kubernetes.io/token	Bootstrap token data

Authentication secrets

Secrets

Builtin Type	Usage
Opaque	arbitrary user-defined data
kubernetes.io/service-account-token	service account token
kubernetes.io/dockercfg	serialized ~/.dockercfg file
kubernetes.io/dockerconfigjson	serialized ~/.docker/config.json file
kubernetes.io/basic-auth	credentials for basic authentication
kubernetes.io/ssh-auth	credentials for SSH authentication
kubernetes.io/tls	data for a TLS client or server
bootstrap.kubernetes.io/token	bootstrap token data



Certificate

Secrets

- Stored as base64 encoded strings
- Not secure as base64 strings are not encrypted
- Should you use secrets?
- Protect them using RBAC authorization policies
- Store secrets elsewhere
 - Cloud providers offer ways to secure these secrets
 - Azure Key Vault
 - AWS Key Management Service
 - Google Cloud KMS
 - HarshiCorp Vault
 - <https://www.vaultproject.io/>

Secrets

```
apiVersion: v1
kind: Secret
metadata:
  name: my-secrets
type: Opaque
data:
  username: am9obmRvZQ==
  password: bXlwYXNzd29yZA==
```

base64 encoded
strings

```
kubectl create secret generic [secretName]
--from-literal=STATE=Michigan
```

Plain text strings

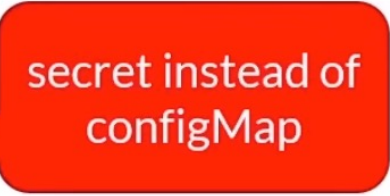
```
apiVersion: v1
kind: Pod
metadata:
  name: pod-example
spec:
  containers:
    - name: test-container
      image: nginx
      env:
        - name: USERNAME
          valueFrom:
            secretKeyRef:
              name: my-secrets
              key: username
```

secretKeyRef
instead of
configMapKeyRef

Secrets – Mounting a Volume

```
apiVersion: v1
kind: Secret
metadata:
  name: my-secrets
type: Opaque
data:
  username: ZXhhbXBsZQ==
  password: bXlwYXNzd29yZA==
```

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-example
spec:
  volumes:
    - name: volmap
      secret:
        secretName: my-secrets
  containers:
    - name: test-container
      image: nginx
      volumeMounts:
        - name: volmap
          mountPath: /etc/secrets
          readOnly: true
```



secret instead of configMap

Secrets – Docker Config

```
apiVersion: v1
kind: Secret
metadata:
  name: secret-dockercfg
type: kubernetes.io/dockercfg
data:
  .dockercfg: "<base64 encoded ~/.dockercfg file>"
```

base64 encoded
Docker config file

```
kubectl create secret docker-registry
secret-dockercfg \
  --docker-username=tiger \
  --docker-password=pass113 \
  --docker-email=tiger@acme.com
```

Registry credentials
in plain text strings

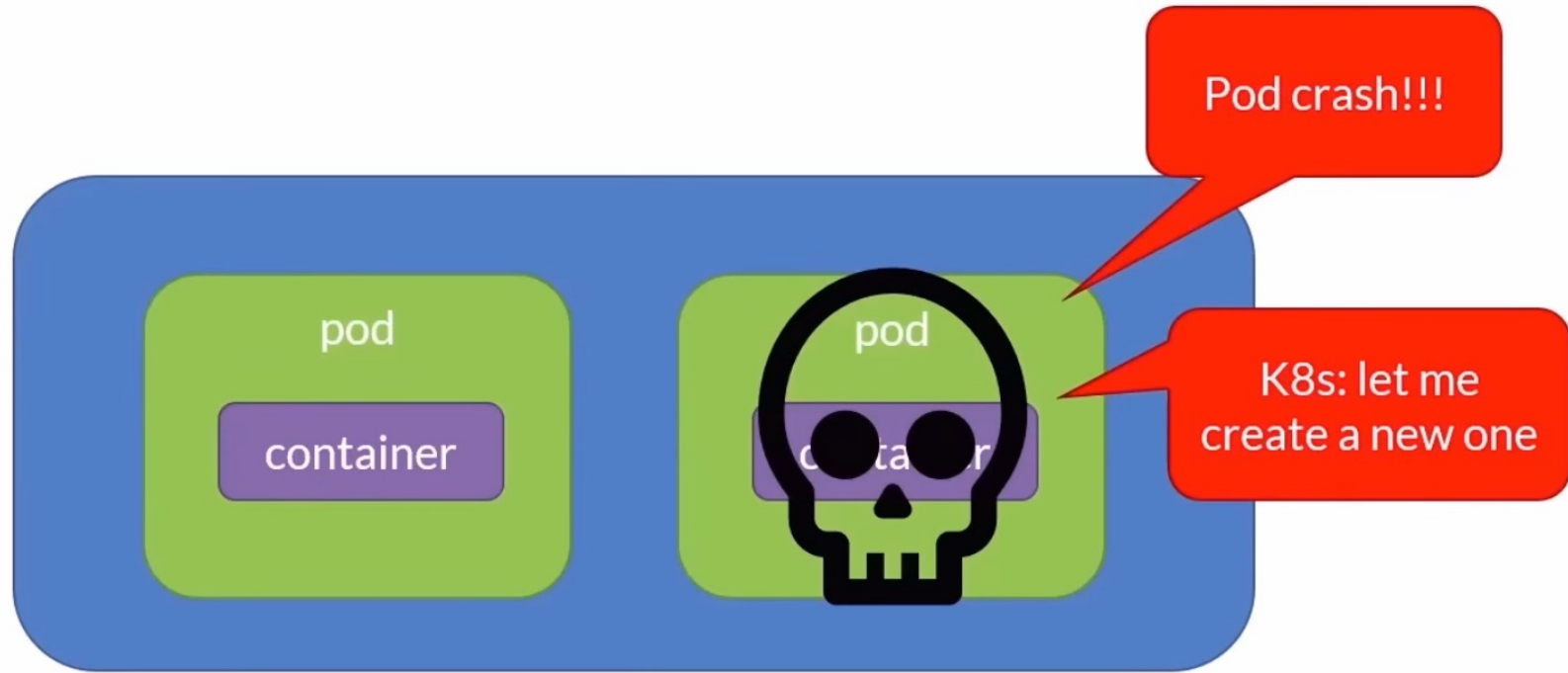
```
apiVersion: v1
kind: Pod
metadata:
  name: private-reg
spec:
  containers:
    - name: private-reg-container
      image: <your-private-image>
      imagePullSecrets:
        - name: secret-dockercfg
```

Provides the credentials
needed to pull the
private image

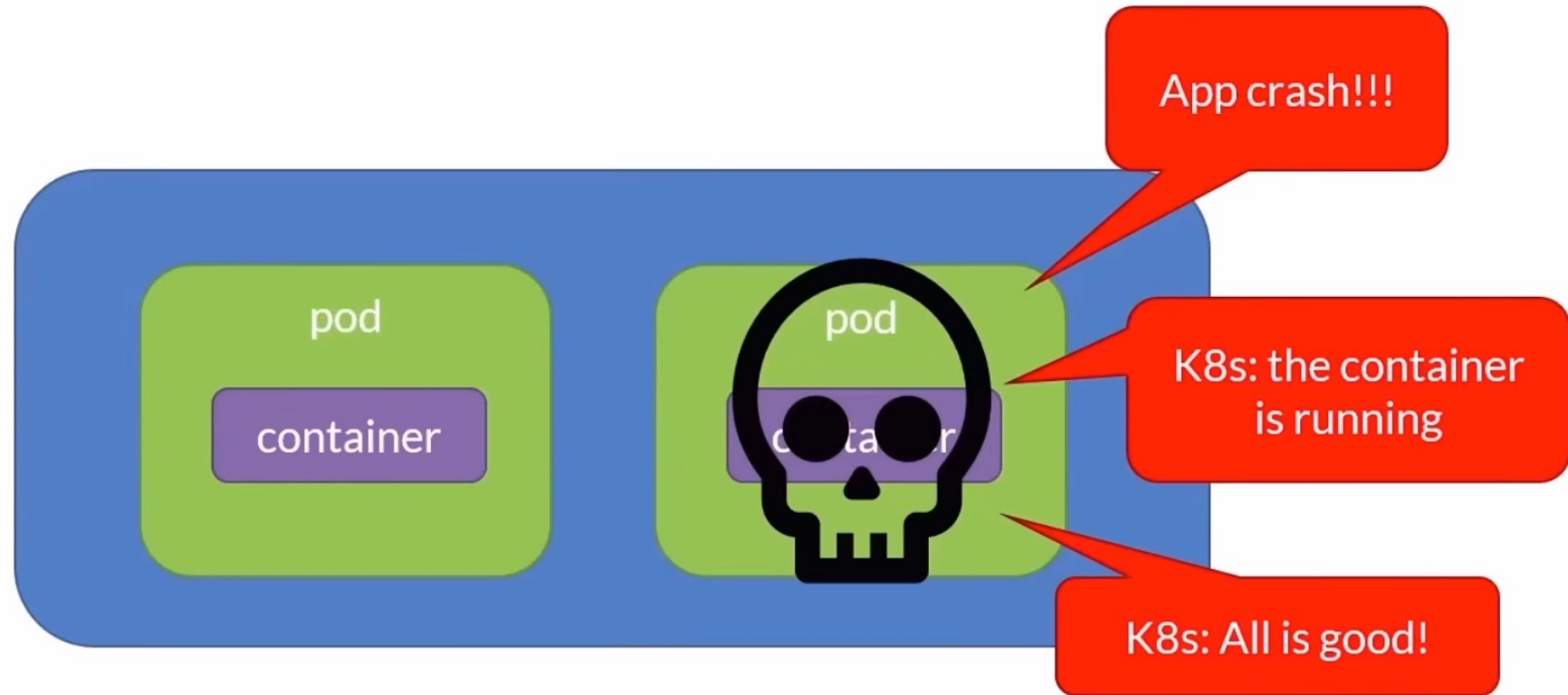
kubectl – Secrets Cheat Sheet

Command	Brief Description
<code>kubectl create secret generic [secretName] --from-literal=state=Michigan</code>	The Imperative Way
<code>kubectl apply -f [secret.yaml]</code>	The declarative way
<code>kubectl get secrets</code>	List the Secrets
<code>kubectl describe secret [secretName]</code>	Get Secret details by its name
<code>kubectl get secrets [secretName] -o YAML</code>	Save a Secret in a YAML file
<code>kubectl delete -f [secret.yaml]</code>	Delete a Secret
<code>kubectl delete secrets [secretName]</code>	Delete a Secret by its name

Pod Crash



App Crash



Observability - Probes

- Startup probes
 - To know when a container has started
- Readiness probes
 - To know when a container is ready to accept traffic
 - A failing readiness probe will stop the application from receiving traffic
- Liveness probes
 - Indicates whether the code is running or not
 - A failing liveness probe will restart the container

Probes

```
apiVersion: v1
kind: Pod
metadata:
  name: goproxy
  labels:
    app: goproxy
spec:
  containers:
  - name: goproxy
    image: k8s.gcr.io/goproxy:0.1
    ports:
    - containerPort: 8080
    readinessProbe:
      tcpSocket:
        port: 8080
      initialDelaySeconds: 5
      periodSeconds: 10
    livenessProbe:
      tcpSocket:
        port: 8080
      initialDelaySeconds: 15
      periodSeconds: 20
    startupProbe:
      httpGet:
        path: /healthz
        port: 80
      failureThreshold: 3
      periodSeconds: 10
```

Startup probe

$3 * 10 = 30$ seconds

Probes

```
apiVersion: v1
kind: Pod
metadata:
  name: goproxy
  labels:
    app: goproxy
spec:
  containers:
  - name: goproxy
    image: k8s.gcr.io/goproxy:0.1
    ports:
    - containerPort: 8080
    readinessProbe:
      tcpSocket:
        port: 8080
      initialDelaySeconds: 5
      periodSeconds: 10
    livenessProbe:
      tcpSocket:
        port: 8080
      initialDelaySeconds: 15
      periodSeconds: 20
    startupProbe:
      httpGet:
        path: /healthz
        port: 80
      failureThreshold: 30
      periodSeconds: 10
```

Readiness probe

Probes

```
apiVersion: v1
kind: Pod
metadata:
  name: goproxy
  labels:
    app: goproxy
spec:
  containers:
  - name: goproxy
    image: k8s.gcr.io/goproxy:0.1
    ports:
    - containerPort: 8080
    readinessProbe:
      tcpSocket:
        port: 8080
      initialDelaySeconds: 5
      periodSeconds: 10
    livenessProbe:
      tcpSocket:
        port: 8080
      initialDelaySeconds: 15
      periodSeconds: 20
    startupProbe:
      httpGet:
        path: /healthz
        port: 80
      failureThreshold: 3
      periodSeconds: 10
```



Liveness probe

Probes

```
apiVersion: v1
kind: Pod
metadata:
  name: goproxy
  labels:
    app: goproxy
spec:
  containers:
  - name: goproxy
    image: k8s.gcr.io/goproxy:0.1
    ports:
    - containerPort: 8080
    readinessProbe:
      tcpSocket:
        port: 8080
      initialDelaySeconds: 5
      periodSeconds: 10
    livenessProbe:
      tcpSocket:
        port: 8080
      initialDelaySeconds: 15
      periodSeconds: 20
    startupProbe:
      httpGet:
        path: /healthz
        port: 80
      failureThreshold: 3
      periodSeconds: 10
```

Stop accepting
traffic

Restart the pod

Will they be in
conflict at some
point?



Probing the container

- The kubelet check periodically the containers using probes
- ExecAction
 - Execute a command inside the container
- TCPSocketAction
 - Check if a TCP socket is open
- HTTPGetAction
 - Performs an HTTP GET against a specific port and path

ExecAction

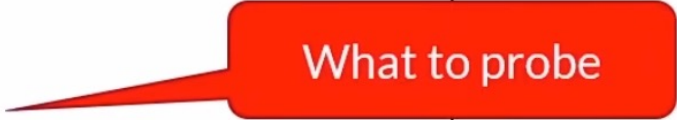
```
apiVersion: v1
kind: Pod
metadata:
  labels:
    test: liveness
  name: liveness-http
spec:
  containers:
    - name: liveness
      image: k8s.gcr.io/liveness
      args:
        - /server
      livenessProbe:
        exec:
          command:
            - cat
            - /tmp/healthy
          initialDelaySeconds: 5
          periodSeconds: 5
```



What to probe

TCPSocketAction

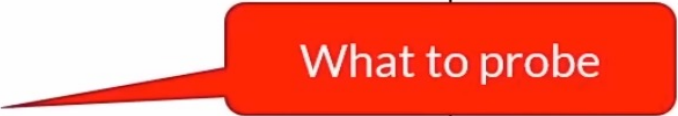
```
apiVersion: v1
kind: Pod
metadata:
  labels:
    test: liveness
    name: liveness-http
spec:
  containers:
  - name: liveness
    image: k8s.gcr.io/liveness
    args:
    - /server
    livenessProbe:
      tcpSocket:
        port: 8080
      initialDelaySeconds: 5
      periodSeconds: 5
```



What to probe

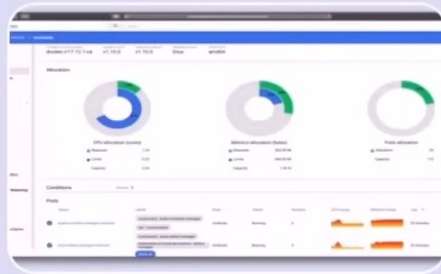
HTTPGetAction

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    test: liveness
    name: liveness-http
spec:
  containers:
  - name: liveness
    image: k8s.gcr.io/liveness
    args:
    - /server
    livenessProbe:
      httpGet:
        path: /healthz
        port: 8080
      initialDelaySeconds: 5
      periodSeconds: 3
```



What to probe

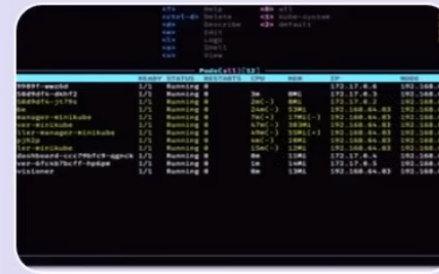
Dashboards



K8s



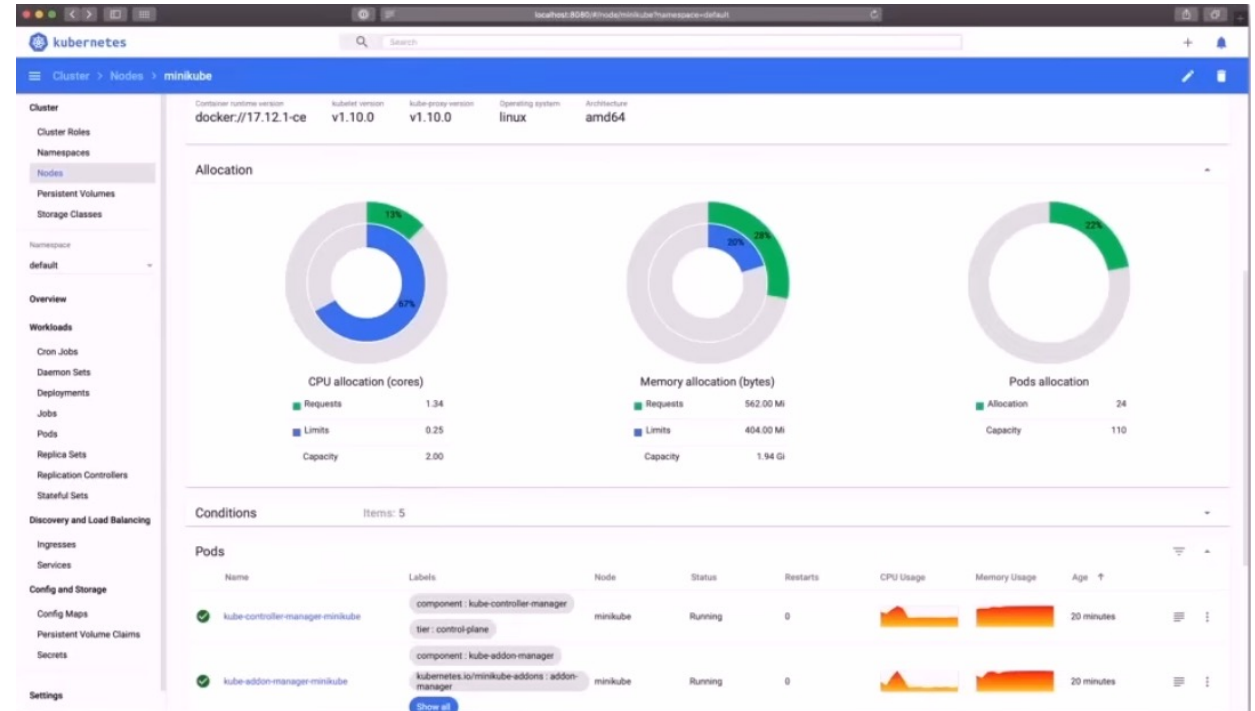
Lens



K9s

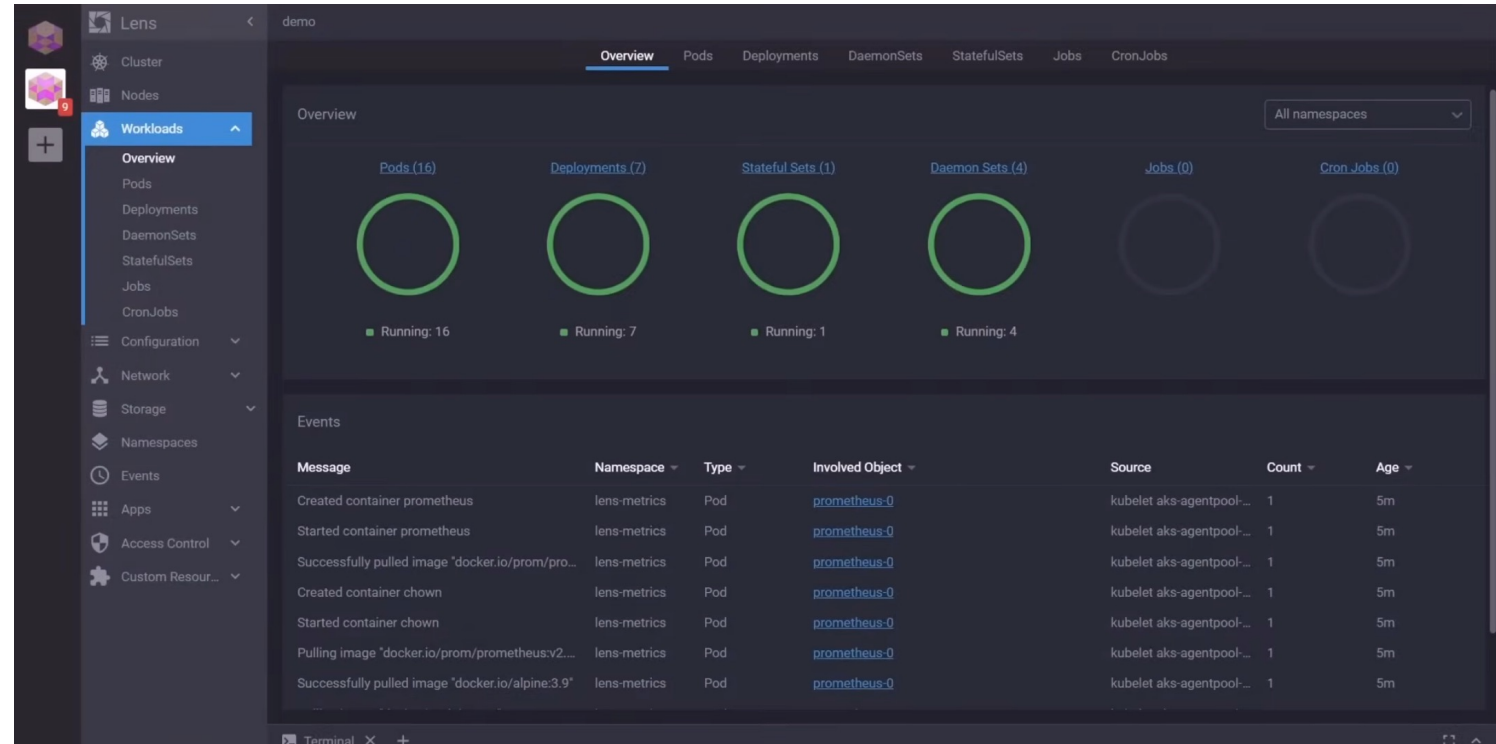
Dashboard - WebUI

- Kubernetes offers a Web UI
- Not Installed by default by
 - Docker Desktop
 - AWS EKS
 - Azure AKS
 - GCP GKE
 - Linode LKE
- Don't install it if not needed



Lens

- Kubernetes IDE
- Runs locally
 - Mac, Windows, Linux
- Built-in terminal
- Maintained by Mirantis
 - <https://lenshq.io/>



Dashboard – K9s

- Dashboard in a terminal
- Windows
 - choco install k9s
- Mac
 - brew install k9s
- Linux
 - <https://github.com/derailed/k9s>

```
Work
Context: do-tor1-k8s
Cluster: do-tor1-k8s
User: do-tor1-k8s-admin
K9s Rev: v0.23.10
K8s Rev: v1.19.3
CPU: 57%
MEM: 55%

<0> all
<1> default

<a> Attach
<ctrl-d> Delete
<d> Describe
<e> Edit
<?> Help
<ctrl-k> Kill

<l> Logs
<p> Logs Previous
<shift-f> Port-Forward
<s> Shell
Show PortForward
YAML

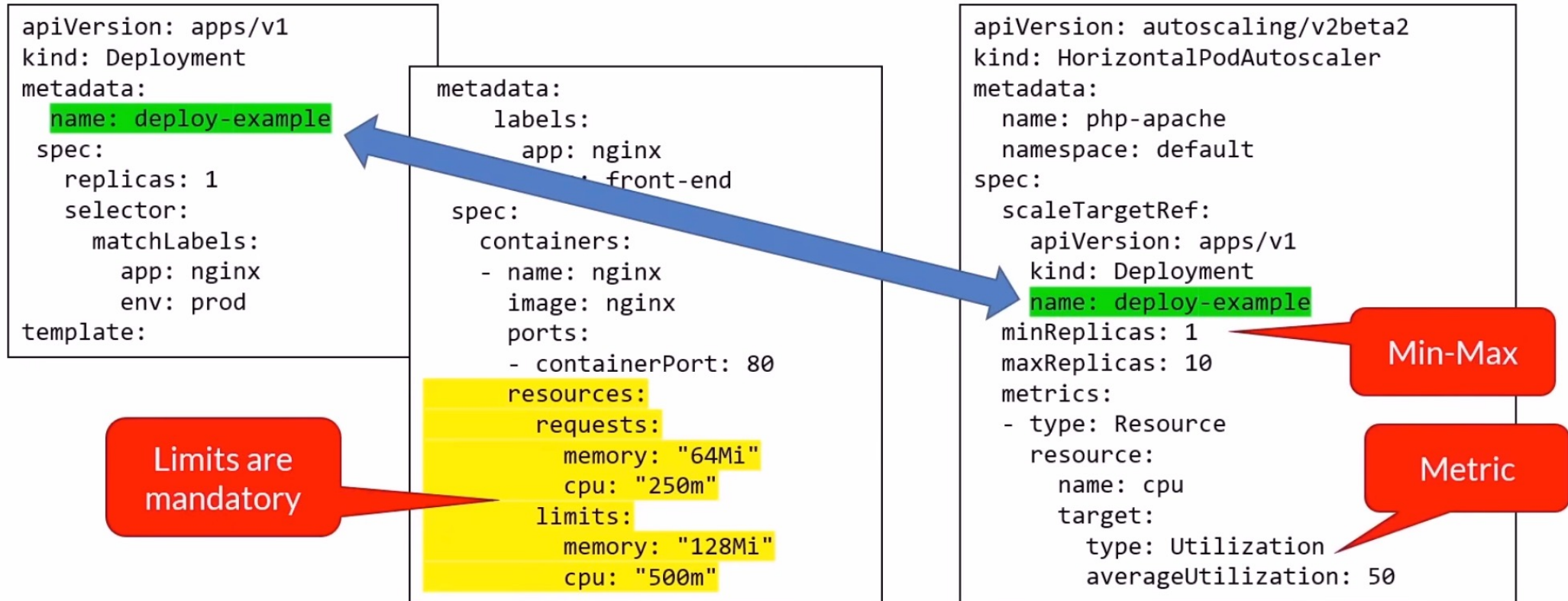
NAME          PF  READY  RESTARTS  STATUS  CPU  MEM  %CPU/R  %CPU/L  %MEM/R  %MEM/L  IP          NODE                                AGE
hello-deploy-58c9bc4b74-2zqwf  ●  1/1      0    Running   1    1      4        2        1        1    10.244.0.61  pool-zb4jihyv0-3hw0b             16m
hello-deploy-58c9bc4b74-kf75s  ●  1/1      0    Running   1    1      4        2        1        1    10.244.0.116 pool-zb4jihyv0-3hw0b             16m

<pod>
```

Scaling – Horizontal Pod AutoScaling (HPA)

- Uses the K8s Metrics Server
- Pods must have requests and limits defined
- The HPA checks the Metrics Server every 30 seconds
- Scale according to the min and max number of replicas defined
- Cooldown/Delay
 - Prevent racing conditions
 - Once a change has been made, HPA waits
 - By default, the delay on scale up events is 3 minutes, and the delay on scale down events is 5 minutes

Scaling – Horizontal Pod AutoScaling (HPA)



kubectl – HPA Cheat Sheet

Command	Brief Description
<code>kubectl autoscale deployment [hpaName] --cpu-percent=50 --min=3 --max=10</code>	The Imperative Way
<code>kubectl apply -f [hpa.yaml]</code>	The declarative way
<code>kubectl get hpa [hpaName]</code>	Get the autoscaler status
<code>kubectl delete -f [hpa.yaml]</code>	Delete the HPA
<code>kubectl delete hpa [hpaName]</code>	Delete the HPA by its name

Conclusion

K8s Ninja!!